

PRACTICA DE LABORATORIO

Interfaz Gráfica de Usuario con Java Swing

Asignatura: Programación Visual (Java)
Unidad: 2 - Componentes Swing
Nivel: Licenciatura / Ingeniería en Sistemas
Duración estimada: 3 - 4 horas
Nombre: Andrea García Espinoza
No. Control: 24580011

Objetivo General

Desarrollar aplicaciones de escritorio con interfaz gráfica de usuario (GUI) utilizando la biblioteca Java Swing, aplicando los componentes visuales fundamentales, el manejo de eventos y las buenas prácticas de programación orientada a objetos.

Competencias a Desarrollar

- Crear y configurar ventanas con JFrame como base de las aplicaciones GUI.
- Mostrar información al usuario usando JLabel.
- Capturar eventos de clic con JButton mediante la interfaz ActionListener.
- Leer datos de texto de una línea con JTextField y múltiples líneas con JTextArea.
- Ofrecer opciones al usuario con JComboBox.
- Construir menús de navegación con JMenuBar, JMenu y JMenuItem.
- Implementar selecciones múltiples con JCheckBox y únicas con JRadioButton.

Material Necesario

- JDK 17 o superior (se recomienda JDK 21 LTS)
- IDE: IntelliJ IDEA Community, Eclipse o NetBeans
- Sistema operativo: Windows, Linux o macOS

NOTA: Todos los ejemplos de esta práctica son proyectos de un solo archivo .java. No se requiere ninguna dependencia externa.

1. Introducción a Java Swing

Java Swing es la biblioteca estándar de Java para construir interfaces graficas de usuario (GUI) en aplicaciones de escritorio. Forma parte del paquete javax.swing y ofrece un conjunto completo de componentes visuales completamente escritos en Java (lightweight), lo que garantiza comportamiento consistente en cualquier plataforma.

1.1 Diferencias entre AWT y Swing

Característica	AWT	Swing
Renderizado	Depende del SO (heavyweight)	100% Java (lightweight)
Apariencia	Nativa de cada SO	Personalizable (Look & Feel)
Componentes	Básicos	Ricos y extensibles
Portabilidad	Variaciones entre SO	Uniforme en todos

1.2 Estructura Base de una Aplicación Swing

Todo programa con interfaz gráfica en Swing sigue esta estructura general:

```
import javax.swing.*;
import java.awt.event.*;

public class MiVentana extends JFrame implements ActionListener {

    // 1. Declarar los componentes como atributos de clase
    private JButton boton1;
    private JLabel label1;

    // 2. Constructor: crear, configurar y agregar componentes
    public MiVentana() {
        setLayout(null);          // Posicionamiento absoluto
        setTitle("Mi Aplicacion"); // Titulo de la ventana

        label1 = new JLabel("Hola, Swing!");
        label1.setBounds(20, 20, 200, 30); // (x, y, ancho, alto)
        add(label1);

        boton1 = new JButton("Click aqui");
        boton1.setBounds(20, 70, 130, 35);
        boton1.addActionListener(this);
        add(boton1);
    }

    // 3. Implementar el metodo de la interfaz ActionListener
    @Override
    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == boton1) {
            label1.setText("Boton presionado!");
        }
    }
}
```

```
// 4. Metodo main: punto de entrada
public static void main(String[] args) {
    MiVentana ventana = new MiVentana();
    ventana.setBounds(100, 100, 320, 200);
    ventana.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    ventana.setVisible(true);
}
}
```

BUENA PRACTICA: Agrega siempre `setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)` en el main para que la aplicacion se cierre correctamente al pulsar la X de la ventana.

1.- Swing - JFrame

El componente básico que se requiere cada vez que se implemente una interfaz visual con la librería Swing es la clase `JFrame`. Esta clase encapsula una ventana clásica de cualquier sistema operativo con entorno gráfico (Windows, OS X, Linux etc.) y se encuentra en el paquete `javax.swing` por lo que es necesario importarla antes de utilizarla.

Ejemplo 1:

Crear una ventana de 300x200 pixeles que se encuentre ubicada en la fila y columna 10:

```
import javax.swing.JFrame;
public class Formulario {
    public static void main(String[] ar) {
        JFrame f=new JFrame();
        f.setBounds(10,10,300,200);
        f.setVisible(true);
    }
}
```

Se importa la clase `JFrame` del paquete `javax.swing`

Se crea la clase `Formulario`

En el main se define y crea un objeto de la clase `JFrame`, llamando luego a los objetos `setBounds` (define la ubicación y tamaño de la ventana) y `setVisible` (determina la visualización de la ventana)

Pero esta forma de trabajar con la clase `JFrame` es de poca utilidad ya que rara vez necesitemos implementar una aplicación que muestre una ventana vacía. Lo más correcto es plantear una clase que herede de la clase `JFrame` y extienda sus responsabilidades a fin de que se pueda agregar botones, etiquetas, editores de línea etc.

Entonces la estructura básica que se empleará para crear una interfaz visual será:

```
import javax.swing.*;
public class Formulario extends JFrame{
    public Formulario() {
        setLayout(null);
    }
    public static void main(String[] ar) {
        Formulario formulario1=new Formulario();
        formulario1.setBounds(10,20,400,300);
        formulario1.setVisible(true);
    }
}
```

Por precaución, se importan todas las clases del paquete `javax.swing`

Se plantea una clase que herede de la clase `JFrame`. En el constructor se define que se ubicarán los controles visuales con coordenadas absolutas mediante la desactivación del Layout heredado

En el main se crea un objeto de la clase y se llaman a los métodos `setBounds` y `setVisible`. `setBounds` ubica la ventana en la columna 10 fila 20 con un ancho de 400 pixeles y alto de 300 pixeles

Ejercicio

Crear una ventana de 1024 x 800 pixeles y que no permita al operador modificar su tamaño.

```
import javax.swing.*;
public class Formulario extends JFrame{
    Formulario() {
        setLayout(null);
    }
    public static void main(String[] ar) {
        Formulario formulario1=new Formulario();
        formulario1.setBounds(0,0,1024,800);
        formulario1.setResizable(false);
        formulario1.setVisible(true);
    }
}
```

En el main adicionalmente a los métodos `setBounds` y `setVisible` que definen la ubicación y tamaño de la ventana, se determina que la ventana no puede ser modificada a través del método `setResizable`

2. Práctica 1: JFrame - La Ventana Principal

2.1 Teoría

JFrame es el contenedor raíz de toda aplicación Swing de escritorio. Encapsula la ventana del sistema operativo y hereda de la clase Frame de AWT. Los métodos más utilizados son:

Metodo	Descripcion
<code>setBounds(x, y, w, h)</code>	Posicion en pantalla y tamaño en píxeles
<code>setVisible(true)</code>	Hace visible la ventana
<code>setTitle("texto")</code>	Establece el título de la barra superior
<code>setResizable(false)</code>	Impide que el usuario redimensione la ventana
<code>setDefaultCloseOperation(...)</code>	Define que ocurre al cerrar (EXIT_ON_CLOSE recomendado)
<code>setSize(ancho, alto)</code>	Cambia el tamaño en tiempo de ejecución

2.2 Ejercicio Guiado

Crea una ventana de 800x600 píxeles, centrada en la pantalla, con título personalizado y que no permita redimensionarse.

```
import javax.swing.*;

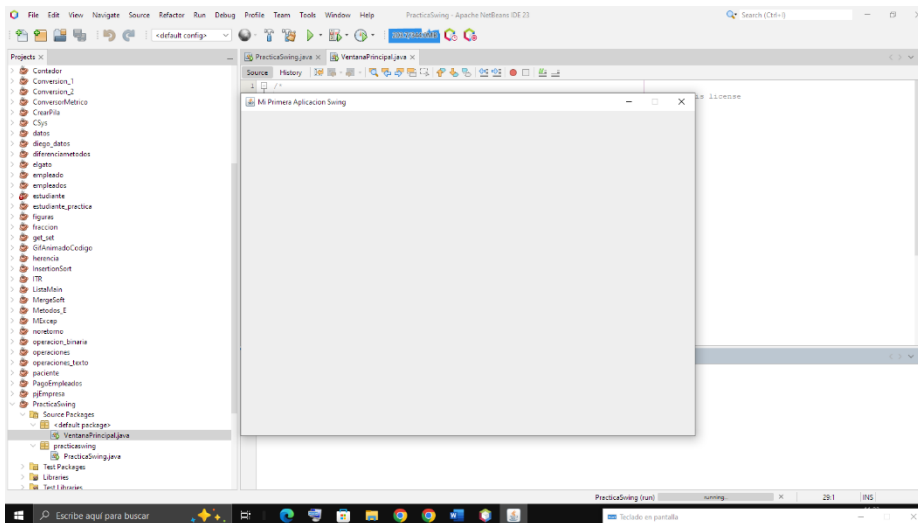
public class VentanaPrincipal extends JFrame {

    public VentanaPrincipal() {
        setLayout(null);
        setTitle("Mi Primera Aplicacion Swing");
        setResizable(false);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }

    public static void main(String[] args) {
        VentanaPrincipal ventana = new VentanaPrincipal();
        // Centrar en pantalla:
        int screenW =
            java.awt.Toolkit.getDefaultToolkit().getScreenSize().width;
        int screenH =
            java.awt.Toolkit.getDefaultToolkit().getScreenSize().height;
        ventana.setBounds((screenW - 800) / 2, (screenH - 600) / 2, 800, 600);
        ventana.setVisible(true);
    }
}
```

Actividades

1. Ejecuta el programa y observa la ventana centrada en pantalla.



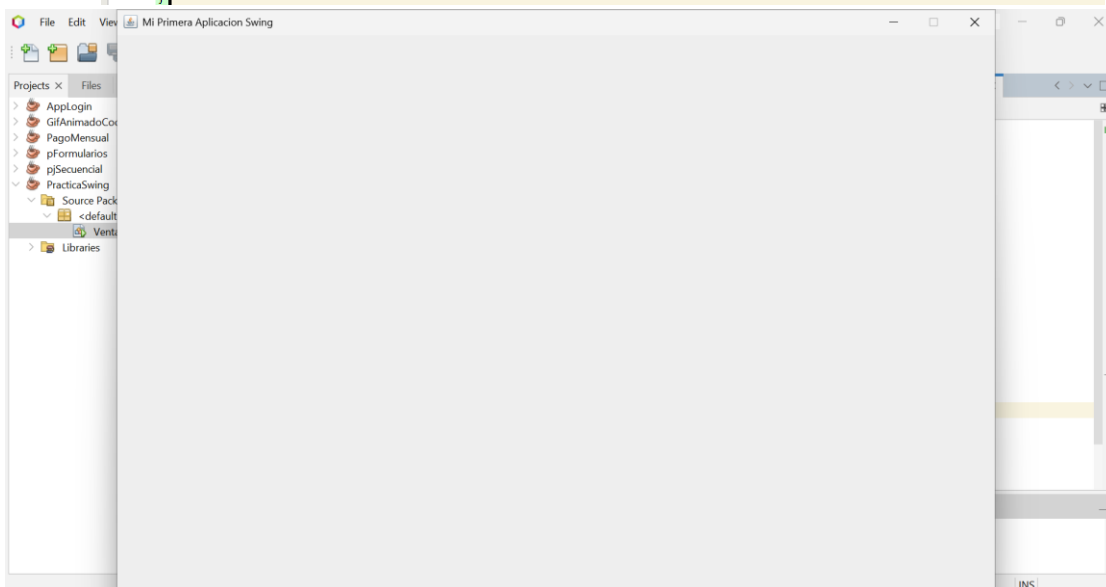
2. Cambia las dimensiones a 1024x768 y prueba de nuevo.

```
import javax.swing.*;

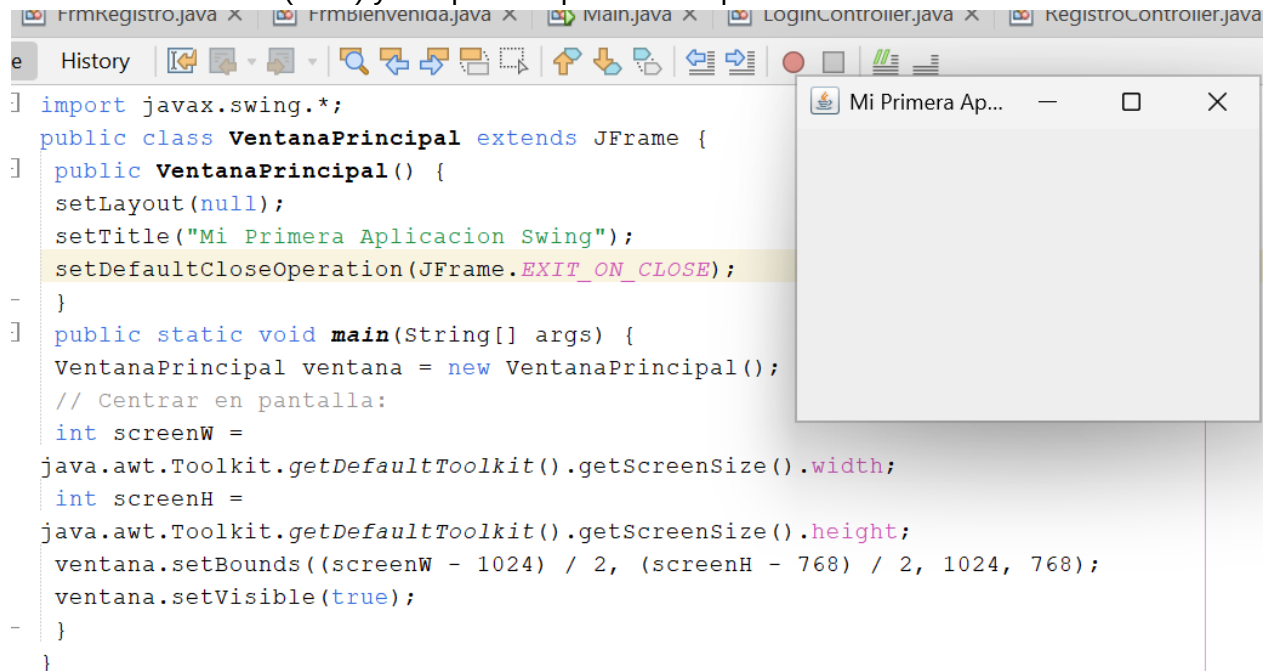
public class VentanaPrincipal extends JFrame {

    public VentanaPrincipal() {
        setLayout(null);
        setTitle("Mi Primera Aplicacion Swing");
        setResizable(false);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }

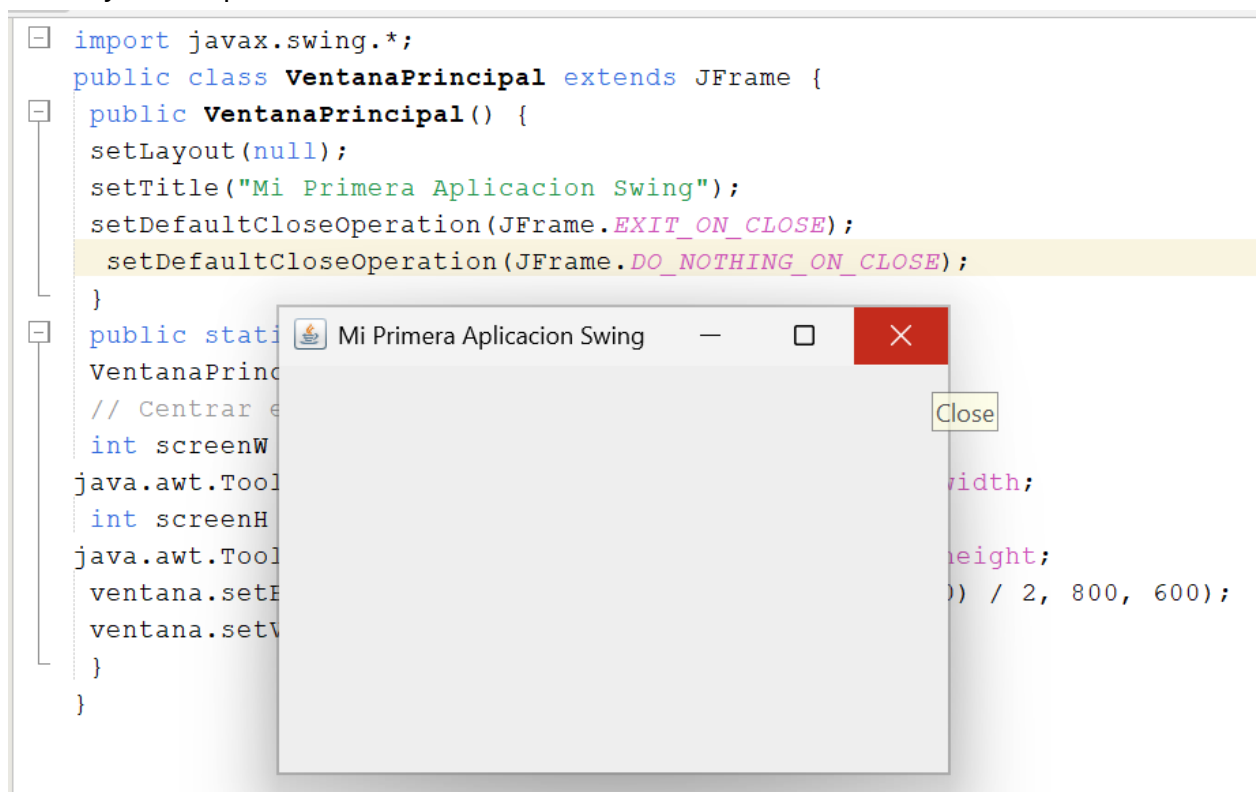
    public static void main(String[] args) {
        VentanaPrincipal ventana = new VentanaPrincipal();
        // Centrar en pantalla:
        int screenW =
            java.awt.Toolkit.getDefaultToolkit().getScreenSize().width;
        int screenH =
            java.awt.Toolkit.getDefaultToolkit().getScreenSize().height;
        ventana.setBounds((screenW - 1024) / 2, (screenH - 768) / 2, 1024, 768);
        ventana.setVisible(true);
    }
}
```



3. Elimina `setResizable(false)` y comprueba que ahora si puedes redimensionar la ventana.



4. Agrega `setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE)` y observa que ya no se puede cerrar la ventana.



3. Práctica 2: JLabel y JTextField

3.1 JLabel - Mostrar Texto

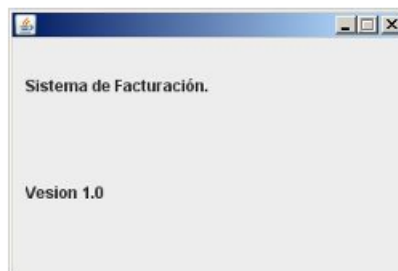
JLabel muestra texto (o una imagen) de forma estatica. Sirve como etiqueta descriptiva para otros controles. Se puede personalizar con metodos como `setFont()`, `setForeground()` y `setHorizontalAlignment()`.

2- Swing – JLabel

La clase JLabel permite mostrar básicamente un texto.

Ejemplo 2:

Confeccionar una ventana que muestre el nombre de un programa en la parte superior y su número de versión en la parte inferior. No se debe permitir modificar el tamaño de la ventana en tiempo de ejecución.



Programa:

```
import javax.swing.*;
public class Formulario extends JFrame {
    private JLabel label1, label2;
    public Formulario() {
        setLayout(null);
        label1=new JLabel("Sistema de Facturación.");
        label1.setBounds(10,20,300,30);
        add(label1);
        label2=new JLabel("Version 1.0");
        label2.setBounds(10,100,100,30);
        add(label2);
    }

    public static void main(String[] ar) {
        Formulario formulario1=new Formulario();
        formulario1.setBounds(0,0,300,200);
        formulario1.setResizable(false);
        formulario1.setVisible(true);
    }
}
```

Se importa el paquete javax.swing donde se encuentran definidas las clases JFrame y JLabel

La clase Formulario hereda de la clase JFrame

Se definen 2 atributos de la clase JLabel

En el constructor Formulario(), se crean los dos JLabel y se ubican utilizando el método setBounds. Además, mediante el método add se añade el JLabel al JFrame. Esto para cada JLabel creado

En el main:

- se crea un objeto de la clase Formulario.
- se invoca al setBounds para ubicar y dar tamaño al JFrame
- Se llama al método setResizable para que no permita la modificación del JFrame en tiempo de ejecución.
- Se usa el método setVisible para que se visualice el JFrame

Ejercicio

Crear tres objetos de la clase JLabel, ubicarlos uno debajo de otro y mostrar nombres de colores.

```
import javax.swing.*;

public class Formulario extends JFrame {
    private JLabel label1,label2,label3;

    public Formulario() {
        setLayout(null);
        label1=new JLabel("Rojo");
        label1.setBounds(10,20,100,30);
        add(label1);
        label2=new JLabel("Verde");
        label2.setBounds(10,60,100,30);
        add(label2);
        label3=new JLabel("Azul");
        label3.setBounds(10,100,100,30);
        add(label3);
    }

    public static void main(String[] ar) {
        Formulario formulario1=new Formulario();
        formulario1.setBounds(0,0,300,200);
        formulario1.setVisible(true);
    }
}
```



3.2 JTextField - Entrada de Texto de Una Línea

JTextField permite al usuario ingresar o editar una cadena de texto en una sola línea. Es el equivalente gráfico de Scanner para capturar datos del teclado. El método getText() recupera el valor ingresado.

3.3 Ejercicio: Formulario de Registro

Crea un formulario que solicite nombre, apellido y edad. Al presionar el boton "Registrar", muestra los datos en el titulo de la ventana.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class FormularioRegistro extends JFrame implements ActionListener {

    private JLabel lblNombre, lblApellido, lblEdad, lblTitulo;
    private JTextField tfNombre, tfApellido, tfEdad;
    private JButton btnRegistrar;

    public FormularioRegistro() {
        setLayout(null);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        // Titulo decorativo en la ventana
        lblTitulo = new JLabel("Formulario de Registro");
        lblTitulo.setFont(new Font("Arial", Font.BOLD, 16));
        lblTitulo.setBounds(100, 15, 280, 30);
        add(lblTitulo);

        lblNombre = new JLabel("Nombre:");
        lblNombre.setBounds(20, 65, 100, 25);
        add(lblNombre);

        tfNombre = new JTextField();
        tfNombre.setBounds(130, 65, 220, 25);
        add(tfNombre);

        lblApellido = new JLabel("Apellido:");
        lblApellido.setBounds(20, 105, 100, 25);
        add(lblApellido);

        tfApellido = new JTextField();
        tfApellido.setBounds(130, 105, 220, 25);
        add(tfApellido);

        lblEdad = new JLabel("Edad:");
        lblEdad.setBounds(20, 145, 100, 25);
        add(lblEdad);

        tfEdad = new JTextField();
        tfEdad.setBounds(130, 145, 80, 25);
        add(tfEdad);

        btnRegistrar = new JButton("Registrar");
        btnRegistrar.setBounds(130, 195, 120, 35);
        btnRegistrar.addActionListener(this);
        add(btnRegistrar);
    }

    @Override
    public void actionPerformed(ActionEvent e) {
```

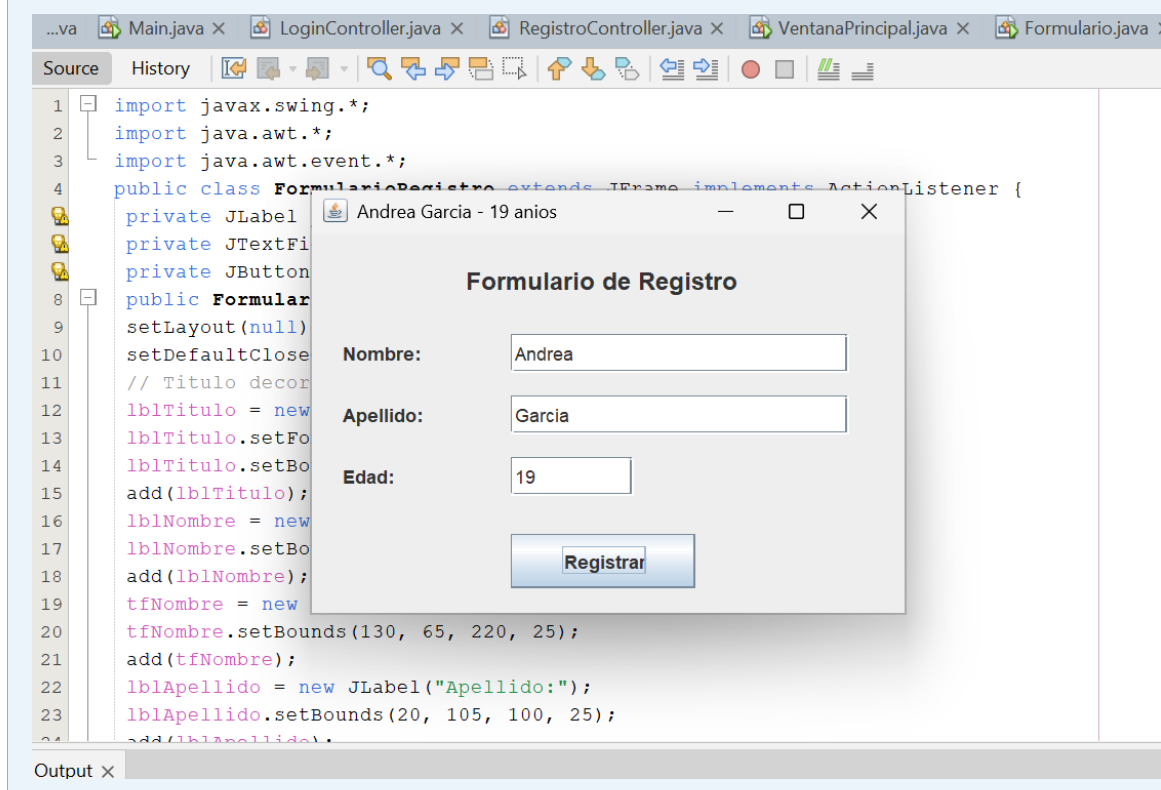
```

        if (e.getSource() == btnRegistrar) {
            String nombre = tfNombre.getText().trim();
            String apellido = tfApellido.getText().trim();
            String edadStr = tfEdad.getText().trim();

            // Validacion basica
            if (nombre.isEmpty() || apellido.isEmpty() || edadStr.isEmpty()) {
                setTitle("ERROR: Completa todos los campos");
                return;
            }
            try {
                int edad = Integer.parseInt(edadStr);
                setTitle(nombre + " " + apellido + " - " + edad + " años");
            } catch (NumberFormatException ex) {
                setTitle("ERROR: La edad debe ser un numero entero");
            }
        }
    }

    public static void main(String[] args) {
        FormularioRegistro f = new FormularioRegistro();
        f.setBounds(200, 200, 400, 290);
        f.setVisible(true);
    }
}

```



NUEVO: Observa el uso de `.trim()` para eliminar espacios al inicio/final, la validación de campos vacíos con `isEmpty()` y el manejo de excepciones con `try-catch` para convertir texto a número. Estas son buenas prácticas esenciales.

Actividades

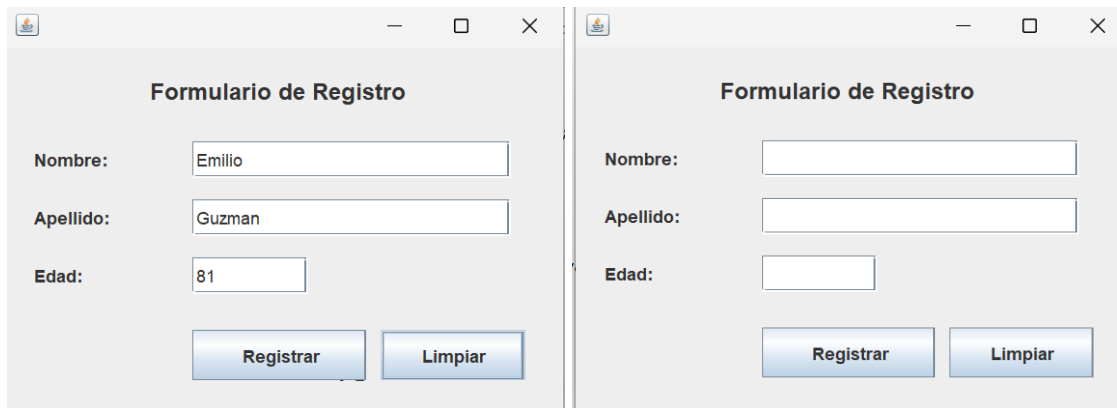
5. Prueba dejando campos vacíos y verifica el mensaje de error.

6. Ingresas texto en el campo Edad y observa el mensaje de excepción.

7. Modifica el programa para que muestre los datos en un JLabel dentro de la ventana (no en el titulo).

```
private JLabel lblResultado;
public FormularioRegistro() {
    setLayout(null);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    lblResultado = new JLabel("");
    lblResultado.setBounds(20, 35, 350, 25);
    add(lblResultado);
}
```

8. Agrega un botón "Limpiar" que borre el contenido de los tres JTextField.



```

btnLimpiar = new JButton("Limpiar");
btnLimpiar.setBounds(260, 195, 100, 35);
btnLimpiar.addActionListener(this);
add(btnLimpiar);
}
@Override
public void actionPerformed(ActionEvent e) {
    if (e.getSource() == btnLimpiar) {
        tfNombre.setText("");
        tfApellido.setText("");
        tfEdad.setText("");
    }
}

```

4. Practica 3: JTextArea - Texto Multilinea

4.1 Teoría

JTextArea permite la entrada y visualización de texto en múltiples líneas. Siempre debe envolverse en un JScrollPane para que aparezcan barras de desplazamiento cuando el contenido supere el área visible. El método `getText()` recupera todo el texto y `setText()` lo establece.

```

// Forma correcta de usar JTextArea con scroll:
JTextArea areaNota = new JTextArea();
areaNota.setLineWrap(true); // Ajuste automatico de linea
areaNota.setWrapStyleWord(true); // Rompe por palabras, no a mitad

JScrollPane scroll = new JScrollPane(areaNota);
scroll.setBounds(10, 50, 400, 300);
add(scroll); // Se agrega el scroll, NO el textarea directamente

```

4.2 Ejercicio: Editor de Notas Simple

Crea un editor de notas que permita ingresar texto, contabilizar palabras y buscar una cadena dentro del texto.

```

import javax.swing.*;
import java.awt.event.*;

public class EditorNotas extends JFrame implements ActionListener {

```

```
private JTextArea areaNota;
private JScrollPane scroll;
private JTextField tfBuscar;
private JButton btnContar, btnBuscar, btnLimpiar;
private JLabel lblEstado;

public EditorNotas() {
    setLayout(null);
    setTitle("Editor de Notas");
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    areaNota = new JTextArea();
    areaNota.setLineWrap(true);
    areaNota.setWrapStyleWord(true);
    scroll = new JScrollPane(areaNota);
    scroll.setBounds(10, 10, 560, 280);
    add(scroll);

    tfBuscar = new JTextField();
    tfBuscar.setBounds(10, 310, 200, 30);
    add(tfBuscar);

    btnBuscar = new JButton("Buscar");
    btnBuscar.setBounds(220, 310, 100, 30);
    btnBuscar.addActionListener(this);
    add(btnBuscar);

    btnContar = new JButton("Contar palabras");
    btnContar.setBounds(330, 310, 140, 30);
    btnContar.addActionListener(this);
    add(btnContar);

    btnLimpiar = new JButton("Limpiar");
    btnLimpiar.setBounds(480, 310, 90, 30);
    btnLimpiar.addActionListener(this);
    add(btnLimpiar);

    lblEstado = new JLabel("Listo.");
    lblEstado.setBounds(10, 360, 560, 25);
    add(lblEstado);
}

@Override
public void actionPerformed(ActionEvent e) {
    String texto = areaNota.getText();

    if (e.getSource() == btnContar) {
        if (texto.trim().isEmpty()) {
            lblEstado.setText("El area esta vacia.");
        } else {
            String[] palabras = texto.trim().split("\\s+");
            lblEstado.setText("Total de palabras: " + palabras.length);
        }
    }

    if (e.getSource() == btnBuscar) {
        String termino = tfBuscar.getText().trim();
    }
}
```

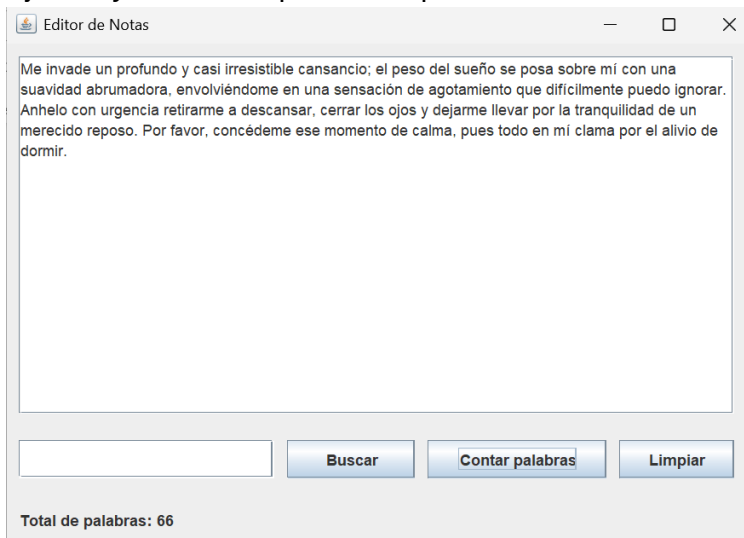
```
        if (termino.isEmpty()) {
            lblEstado.setText("Escribe algo en el campo de busqueda.");
            return;
        }
        if (texto.contains(termino)) {
            lblEstado.setText("Encontrado: '" + termino + "' en el texto.");
        } else {
            lblEstado.setText("No se encontro: '" + termino + "'");
        }
    }

    if (e.getSource() == btnLimpiar) {
        areaNota.setText("");
        tfBuscar.setText("");
        lblEstado.setText("Listo.");
    }
}

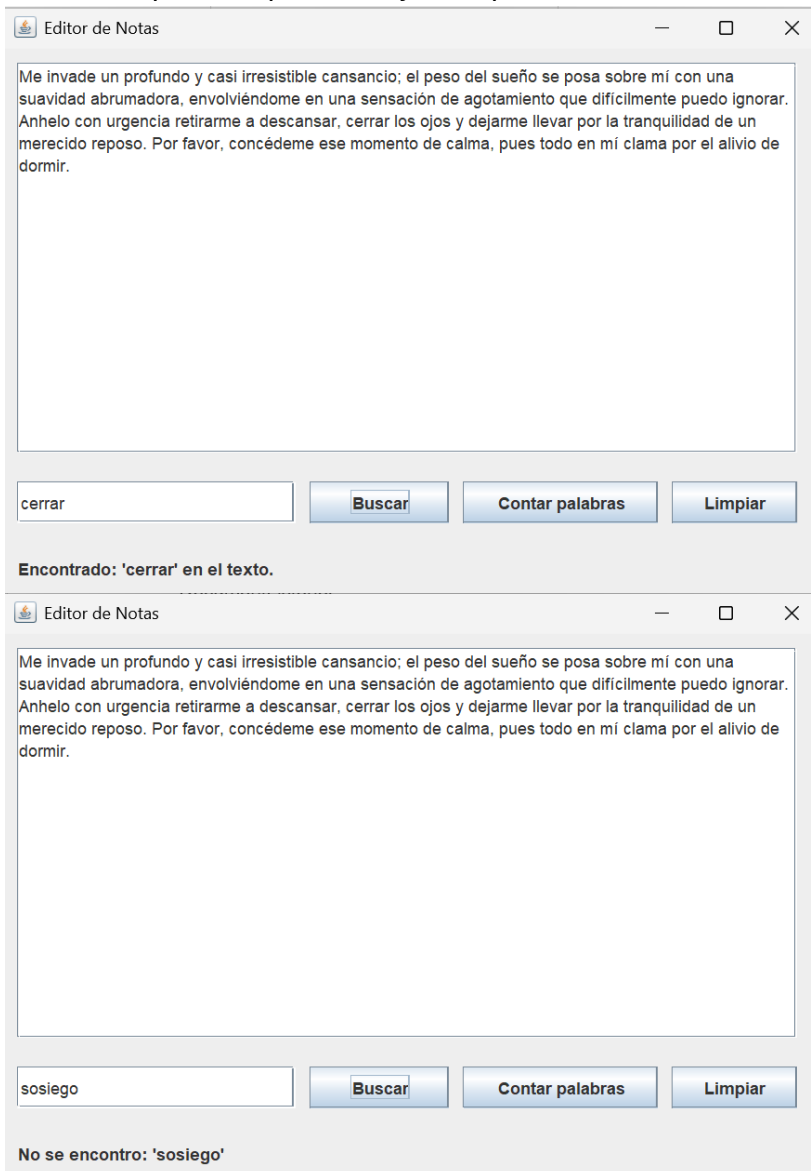
public static void main(String[] args) {
    EditorNotas editor = new EditorNotas();
    editor.setBounds(150, 100, 600, 430);
    editor.setVisible(true);
}
}
```

Actividades

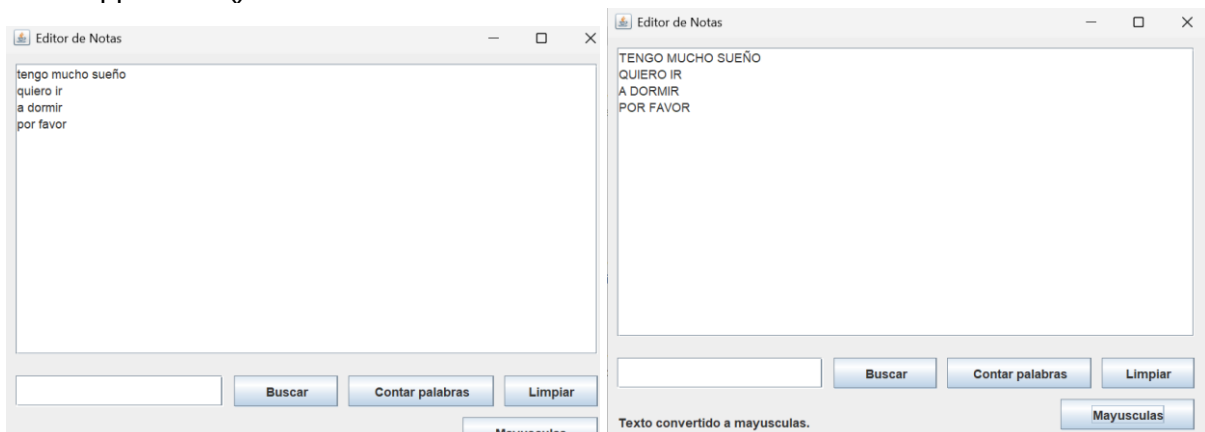
9. Ejecuta y escribe un párrafo de prueba. Verifica el conteo de palabras.



10. Busca una palabra que exista y otra que no exista en el texto.



11. Agrega un botón 'Mayúsculas' que convierta todo el texto a mayúsculas usando toUpperCase().



```
import javax.swing.*;
```

```
import java.awt.event.*;

public class EditorNotas extends JFrame implements ActionListener {
    private JTextArea areaNota;
    private JScrollPane scroll;
    private JTextField tfBuscar;
    private JButton btnContar, btnBuscar, btnLimpiar;
    private JLabel lblEstado;

    // Botón agregado para convertir el texto a mayúsculas
    private JButton btnMayusculas;

    public EditorNotas() {
        setLayout(null);
        setTitle("Editor de Notas");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        areaNota = new JTextArea();
        areaNota.setLineWrap(true);
        areaNota.setWrapStyleWord(true);

        scroll = new JScrollPane(areaNota);
        scroll.setBounds(10, 10, 560, 280);
        add(scroll);

        tfBuscar = new JTextField();
        tfBuscar.setBounds(10, 310, 200, 30);
        add(tfBuscar);

        btnBuscar = new JButton("Buscar");
        btnBuscar.setBounds(220, 310, 100, 30);
        btnBuscar.addActionListener(this);
        add(btnBuscar);

        btnContar = new JButton("Contar palabras");
        btnContar.setBounds(330, 310, 140, 30);
        btnContar.addActionListener(this);
        add(btnContar);

        btnLimpiar = new JButton("Limpiar");
        btnLimpiar.setBounds(480, 310, 90, 30);
        btnLimpiar.addActionListener(this);
        add(btnLimpiar);

        lblEstado = new JLabel("Listo.");
        lblEstado.setBounds(10, 360, 560, 25);
        add(lblEstado);

        // Se crea el botón "Mayusculas" y se agrega a la ventana
        btnMayusculas = new JButton("Mayusculas");
        btnMayusculas.setBounds(440, 350, 130, 30);
        btnMayusculas.addActionListener(this); // Se registra el evento del botón
        add(btnMayusculas);
    }

    @Override
```



```
public void actionPerformed(ActionEvent e) {

    // Evento del botón Mayúsculas
    if (e.getSource() == btnMayusculas) {
        // Se obtiene el texto actual del área
        String texto = areaNota.getText();

        // Se convierte todo el texto a mayúsculas
        areaNota.setText(texto.toUpperCase());

        // Se muestra un mensaje en la etiqueta de estado
        lblEstado.setText("Texto convertido a mayusculas.");
    }

    String texto = areaNota.getText();

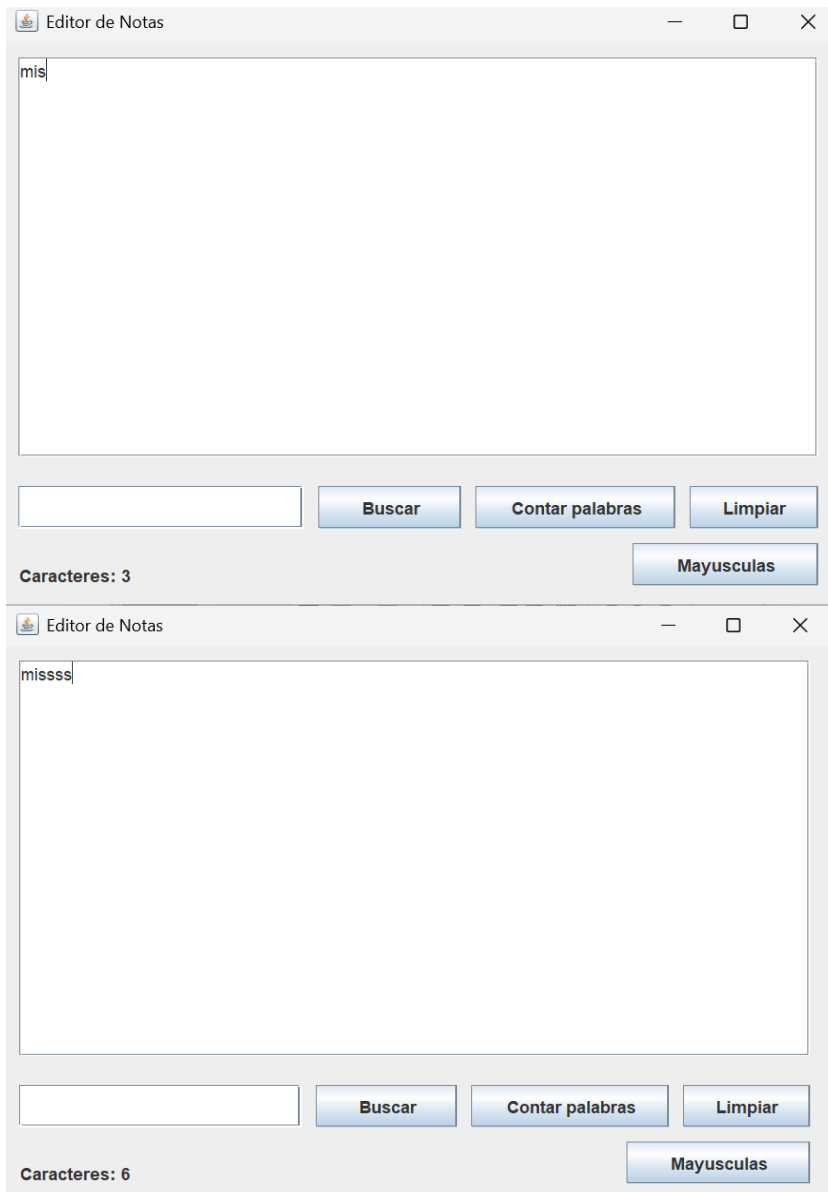
    if (e.getSource() == btnContar) {
        if (texto.trim().isEmpty()) {
            lblEstado.setText("El area esta vacia.");
        } else {
            String[] palabras = texto.trim().split("\\s+");
            lblEstado.setText("Total de palabras: " + palabras.length);
        }
    }

    if (e.getSource() == btnBuscar) {
        String termino = tfBuscar.getText().trim();
        if (termino.isEmpty()) {
            lblEstado.setText("Escribe algo en el campo de busqueda.");
            return;
        }
        if (texto.contains(termino)) {
            lblEstado.setText("Encontrado: " + termino + " en el texto.");
        } else {
            lblEstado.setText("No se encontro: " + termino + "");
        }
    }

    if (e.getSource() == btnLimpiar) {
        areaNota.setText("");
        tfBuscar.setText("");
        lblEstado.setText("Listo.");
    }

    public static void main(String[] args) {
        EditorNotas editor = new EditorNotas();
        editor.setBounds(150, 100, 600, 430);
        editor.setVisible(true);
    }
}
```

12. Agrega un contador de caracteres que se actualice en tiempo real usando un `DocumentListener`.



```
import javax.swing.*;
import java.awt.event.*;
```

```
// Importaciones necesarias para detectar cambios en el texto
import javax.swing.event.DocumentListener;
import javax.swing.event.DocumentEvent;
```

```
public class EditorNotas extends JFrame implements ActionListener {
    private JTextArea areaNota;
    private JScrollPane scroll;
    private JTextField tfBuscar;
    private JButton btnContar, btnBuscar, btnLimpiar;
    private JLabel lblEstado;
    private JButton btnMayusculas;
```

```
    public EditorNotas() {
        setLayout(null);
        setTitle("Editor de Notas");
```

```
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

areaNota = new JTextArea();
areaNota.setLineWrap(true);
areaNota.setWrapStyleWord(true);

// Se agrega un DocumentListener al área de texto
// Esto permite detectar cuando el usuario escribe o borra texto
areaNota.getDocument().addDocumentListener(new DocumentListener() {

    // Se ejecuta cuando se inserta texto
    public void insertUpdate(DocumentEvent e) {
        actualizarContador();
    }

    // Se ejecuta cuando se elimina texto
    public void removeUpdate(DocumentEvent e) {
        actualizarContador();
    }

    // Se ejecuta cuando hay cambios en el formato (no siempre se usa)
    public void changedUpdate(DocumentEvent e) {
        actualizarContador();
    }
});

scroll = new JScrollPane(areaNota);
scroll.setBounds(10, 10, 560, 280);
add(scroll);

tfBuscar = new JTextField();
tfBuscar.setBounds(10, 310, 200, 30);
add(tfBuscar);

btnBuscar = new JButton("Buscar");
btnBuscar.setBounds(220, 310, 100, 30);
btnBuscar.addActionListener(this);
add(btnBuscar);

btnContar = new JButton("Contar palabras");
btnContar.setBounds(330, 310, 140, 30);
btnContar.addActionListener(this);
add(btnContar);

btnLimpiar = new JButton("Limpiar");
btnLimpiar.setBounds(480, 310, 90, 30);
btnLimpiar.addActionListener(this);
add(btnLimpiar);

lblEstado = new JLabel("Listo.");
lblEstado.setBounds(10, 360, 560, 25);
```

```
add(lblEstado);

btnMayusculas = new JButton("Mayusculas");
btnMayusculas.setBounds(440, 350, 130, 30);
btnMayusculas.addActionListener(this);
add(btnMayusculas);
}

// Método creado para actualizar el contador de caracteres
private void actualizarContador() {
    // Se obtiene la cantidad de caracteres del texto
    int caracteres = areaNota.getText().length();

    // Se muestra el total en la etiqueta de estado
    lblEstado.setText("Caracteres: " + caracteres);
}

@Override
public void actionPerformed(ActionEvent e) {

    if (e.getSource() == btnMayusculas) {
        String texto = areaNota.getText();
        areaNota.setText(texto.toUpperCase());
        lblEstado.setText("Texto convertido a mayusculas.");
    }

    String texto = areaNota.getText();

    if (e.getSource() == btnContar) {
        if (texto.trim().isEmpty()) {
            lblEstado.setText("El area esta vacia.");
        } else {
            String[] palabras = texto.trim().split("\\s+");
            lblEstado.setText("Total de palabras: " + palabras.length);
        }
    }

    if (e.getSource() == btnBuscar) {
        String termino = tfBuscar.getText().trim();
        if (termino.isEmpty()) {
            lblEstado.setText("Escribe algo en el campo de busqueda.");
            return;
        }
        if (texto.contains(termino)) {
            lblEstado.setText("Encontrado: '" + termino + "' en el texto.");
        } else {
            lblEstado.setText("No se encontro: '" + termino + "'");
        }
    }

    if (e.getSource() == btnLimpiar) {
```

```
areaNota.setText("");
tfBuscar.setText("");
lblEstado.setText("Listo.");
}
}

public static void main(String[] args) {
    EditorNotas editor = new EditorNotas();
    editor.setBounds(150, 100, 600, 430);
    editor.setVisible(true);
}
}
```

5. Practica 4: JComboBox - Lista Desplegable

5.1 Teoría

JComboBox presenta una lista desplegable donde el usuario puede seleccionar exactamente una opción. Se puede poblar con `addItem()` o con un arreglo pasado al constructor. Para conocer el elemento seleccionado se usa `getSelectedItem()` (devuelve `Object`) o `getSelectedIndex()` (devuelve el índice).

5.2 Ejercicio: Calculadora de Operaciones básicas

Crea una calculadora que permita ingresar dos números, seleccionar la operación con un JComboBox y mostrar el resultado al presionar un botón.

```
import javax.swing.*;
import java.awt.event.*;

public class Calculadora extends JFrame implements ActionListener {

    private JLabel lblNum1, lblNum2, lblOp, lblResultado;
    private JTextField tfNum1, tfNum2;
    private JComboBox<String> cmbOperacion;
    private JButton btnCalcular;

    public Calculadora() {
        setLayout(null);
        setTitle("Calculadora Swing");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        lblNum1 = new JLabel("Numero 1:");
        lblNum1.setBounds(20, 20, 100, 25);
        add(lblNum1);

        tfNum1 = new JTextField();
        tfNum1.setBounds(130, 20, 120, 25);
        add(tfNum1);

        lblNum2 = new JLabel("Numero 2:");
        lblNum2.setBounds(20, 60, 100, 25);
        add(lblNum2);

        tfNum2 = new JTextField();
        tfNum2.setBounds(130, 60, 120, 25);
        add(tfNum2);

        lblOp = new JLabel("Operacion:");
        lblOp.setBounds(20, 100, 100, 25);
        add(lblOp);

        // Uso de generics para tipado seguro
        String[] ops = { "Suma (+)", "Resta (-)", "Multiplicacion (*)",
            "Division (/)" };
        cmbOperacion = new JComboBox<>(ops);
        cmbOperacion.setBounds(130, 100, 180, 25);
```

```
add(cmbOperacion);

btnCalcular = new JButton("Calcular");
btnCalcular.setBounds(130, 145, 120, 35);
btnCalcular.addActionListener(this);
add(btnCalcular);

lblResultado = new JLabel("Resultado: ---");
lblResultado.setBounds(20, 200, 320, 30);
add(lblResultado);
}

@Override
public void actionPerformed(ActionEvent e) {
    try {
        double n1 = Double.parseDouble(tfNum1.getText().trim());
        double n2 = Double.parseDouble(tfNum2.getText().trim());
        int opIndex = cmbOperacion.getSelectedIndex();
        double resultado = 0;

        switch (opIndex) {
            case 0: resultado = n1 + n2; break;
            case 1: resultado = n1 - n2; break;
            case 2: resultado = n1 * n2; break;
            case 3:
                if (n2 == 0) {
                    lblResultado.setText("Error: Division por cero");
                    return;
                }
                resultado = n1 / n2; break;
        }
        lblResultado.setText(String.format("Resultado: %.2f", resultado));
    } catch (NumberFormatException ex) {
        lblResultado.setText("Error: ingresa valores numericos validos");
    }
}

public static void main(String[] args) {
    Calculadora c = new Calculadora();
    c.setBounds(250, 200, 380, 280);
    c.setVisible(true);
}
}
```

NUEVO: Se usan generics `JComboBox<String>` para evitar el cast en `getSelectedItem()`. El metodo `String.format("%.2f", valor)` formatea el resultado con 2 decimales. Se controla la division por cero explicitamente.

Actividades

13. Prueba todas las operaciones con valores positivos y negativos.

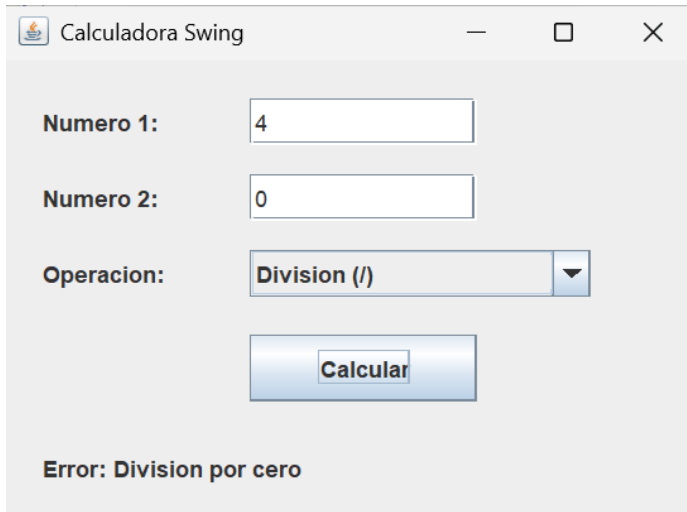
The image displays six screenshots of a Java Swing calculator application, arranged in a 3x2 grid. Each window is titled 'Calculadora Swing' and contains the following elements:

- Two input fields for 'Numero 1' and 'Numero 2'.
- A dropdown menu for 'Operacion' with options: Suma (+), Resta (-), Multiplicacion (*), and Division (/).
- A 'Calcular' button.
- A 'Resultado' label showing the result of the operation.

The results shown in the screenshots are:

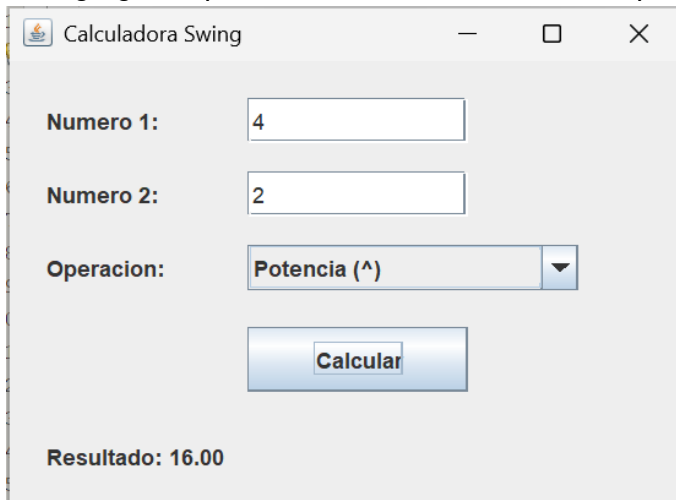
Operacion	Numero 1	Numero 2	Resultado
Suma (+)	2	6	8.00
Resta (-)	2	6	-4.00
Multiplicacion (*)	2	6	12.00
Division (/)	2	6	0.33
Suma (+)	-5	-2	-7.00
Resta (-)	-5	-2	-3.00
Multiplicacion (*)	-5	-2	10.00
Division (/)	-5	-2	2.50

14. Intenta dividir entre cero y verifica el mensaje de error.



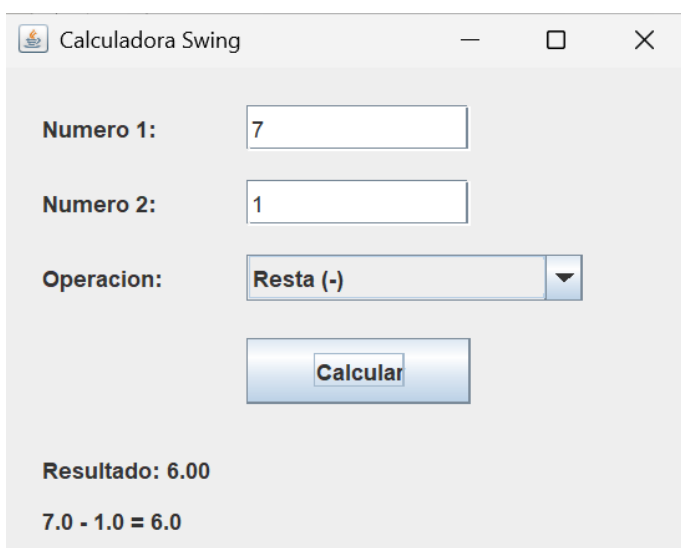
The screenshot shows a Java Swing window titled "Calculadora Swing". It contains two text input fields: "Numero 1:" with the value "4" and "Numero 2:" with the value "0". Below these is a dropdown menu for "Operacion:" set to "Division (/)". A blue "Calcular" button is positioned below the dropdown. At the bottom of the window, the text "Error: Division por cero" is displayed.

15. Agrega la operación 'Potencia' usando `Math.pow(n1, n2)`.



The screenshot shows the "Calculadora Swing" window with "Numero 1:" set to "4" and "Numero 2:" set to "2". The "Operacion:" dropdown is now set to "Potencia (^)". The "Calcular" button is visible. At the bottom, the text "Resultado: 16.00" is displayed.

16. Agrega un JLabel que muestre la operación completa en formato 'n1 op n2 = resultado'.



The screenshot shows the "Calculadora Swing" window with "Numero 1:" set to "7" and "Numero 2:" set to "1". The "Operacion:" dropdown is set to "Resta (-)". The "Calcular" button is visible. At the bottom, the text "Resultado: 6.00" is displayed, followed by a new line showing the full calculation: "7.0 - 1.0 = 6.0".

6. Practica 5: JMenuBar, JMenu y JMenuItem

6.1 Teoría

Los menus en Swing requieren tres tipos de objetos que se relacionan jerárquicamente:

- JMenuBar: la barra horizontal en la parte superior de la ventana. Solo hay una por JFrame.
- JMenu: cada opcion de la barra que al hacer clic despliega un submenu. Puede contener otros JMenu (submenus) o JMenuItem.
- JMenuItem: la opcion final que el usuario puede seleccionar y que dispara un ActionEvent.

6.2 Ejercicio: Aplicación con Menú Completo

Crea una aplicación con barra de menú que incluya opciones para cambiar el color de fondo, ajustar el tamaño de la ventana y salir.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class AppConMenu extends JFrame implements ActionListener {

    private JMenuBar menuBar;
    private JMenu menuVista, menuTamano, menuAyuda;
    private JMenuItem miRojo, miAzul, miVerde, miBlancoFondo;
    private JMenuItem mi800, mi1024, miSalir, miAcerca;
    private JLabel lblInfo;

    public AppConMenu() {
        setLayout(null);
        setTitle("Aplicacion con Menu");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        // ---- Construcccion del menu ----
        menuBar = new JMenuBar();
        setJMenuBar(menuBar);

        // Menu Vista
        menuVista = new JMenu("Vista");
        menuBar.add(menuVista);

        miRojo = new JMenuItem("Fondo Rojo");
        miAzul = new JMenuItem("Fondo Azul");
        miVerde = new JMenuItem("Fondo Verde");
        miBlancoFondo = new JMenuItem("Restaurar fondo");
        miRojo.addActionListener(this);
        miAzul.addActionListener(this);
        miVerde.addActionListener(this);
        miBlancoFondo.addActionListener(this);
        menuVista.add(miRojo);
        menuVista.add(miAzul);
        menuVista.add(miVerde);
        menuVista.addSeparator(); // Linea separadora
```

```

        menuVista.add(miBlancoFondo);

        // Menu Ventana (con submenu)
        menuTamano = new JMenu("Ventana");
        menuBar.add(menuTamano);

        mi800 = new JMenuItem("800 x 600");
        mi1024 = new JMenuItem("1024 x 768");
        miSalir = new JMenuItem("Salir");
        mi800.addActionListener(this);
        mi1024.addActionListener(this);
        miSalir.addActionListener(this);
        menuTamano.add(mi800);
        menuTamano.add(mi1024);
        menuTamano.addSeparator();
        menuTamano.add(miSalir);

        // Menu Ayuda
        menuAyuda = new JMenu("Ayuda");
        menuBar.add(menuAyuda);
        miAcerca = new JMenuItem("Acerca de...");
        miAcerca.addActionListener(this);
        menuAyuda.add(miAcerca);

        // Contenido de la ventana
        lblInfo = new JLabel("Usa el menu para interactuar");
        lblInfo.setBounds(20, 20, 400, 30);
        add(lblInfo);
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        Object src = e.getSource();
        Container fondo = getContentPane();

        if (src == miRojo)        fondo.setBackground(new Color(220, 80, 80));
        else if (src == miAzul)   fondo.setBackground(new Color(70, 130, 200));
        else if (src == miVerde)  fondo.setBackground(new Color(80, 200, 120));
        else if (src == miBlancoFondo) fondo.setBackground(null);
        else if (src == mi800)    setSize(800, 600);
        else if (src == mi1024)  setSize(1024, 768);
        else if (src == miSalir)  System.exit(0);
        else if (src == miAcerca) {
            JOptionPane.showMessageDialog(this,
                "Practica de Menus con Swing\nVersion 1.0",
                "Acerca de", JOptionPane.INFORMATION_MESSAGE);
        }
    }

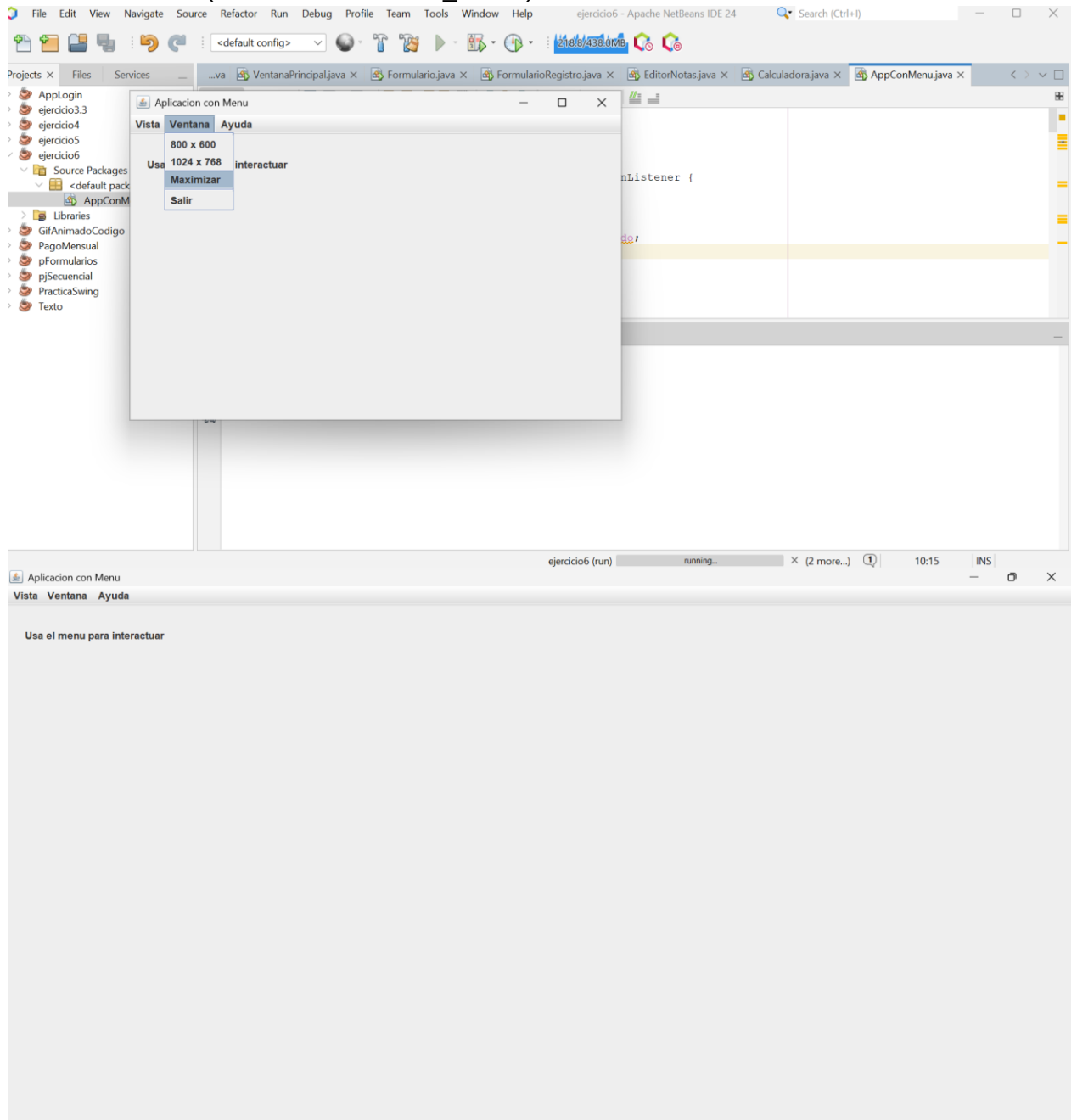
    public static void main(String[] args) {
        AppConMenu app = new AppConMenu();
        app.setBounds(150, 100, 600, 400);
        app.setVisible(true);
    }
}

```

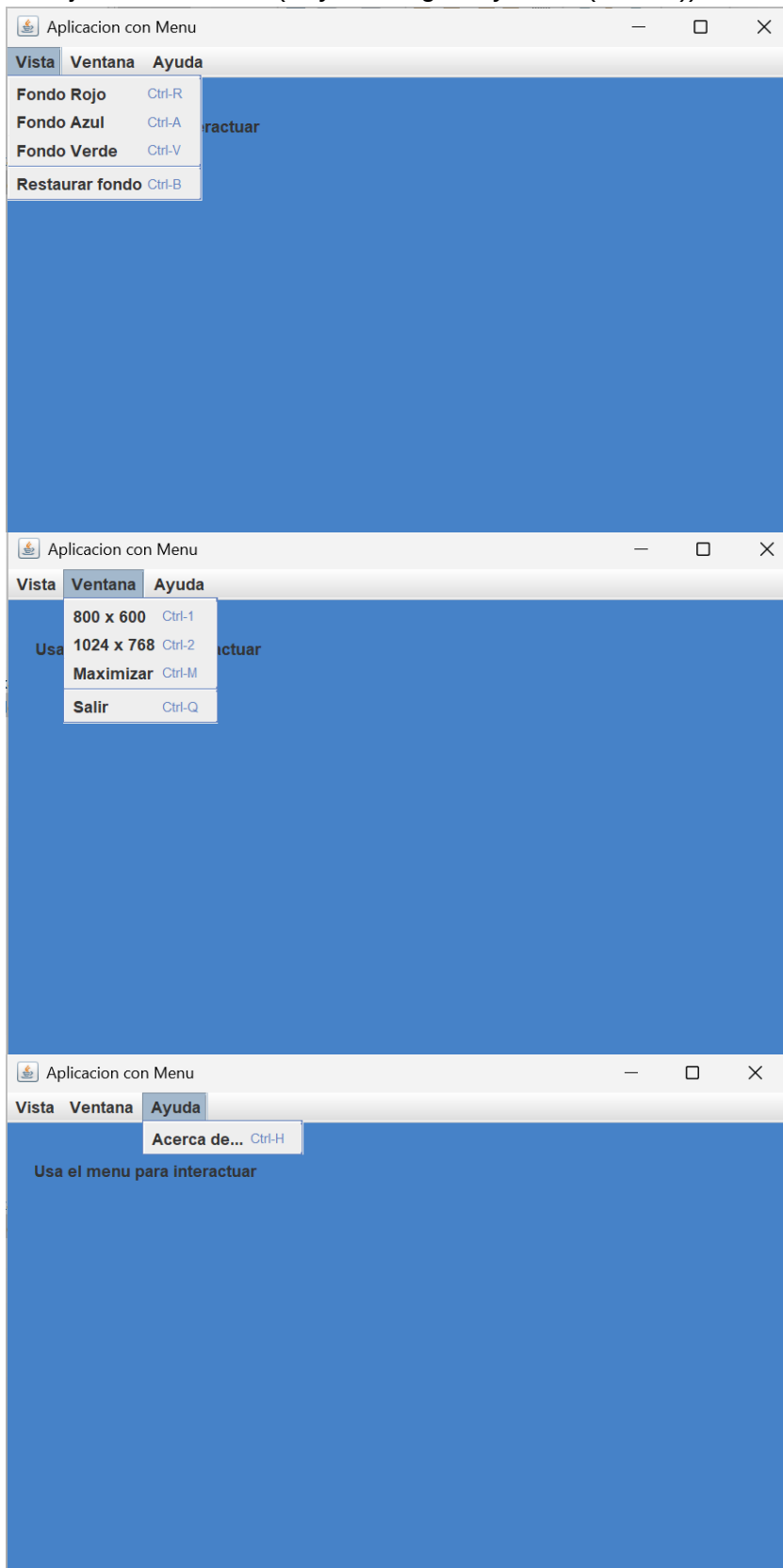
NUEVO: Se introducen `addSeparator()` para lineas divisorias en el menu, y `JOptionPane.showMessageDialog()` para dialogos emergentes informativos. Tambien se usa `if-else if` en lugar de multiples `if` para mayor eficiencia.

Actividades

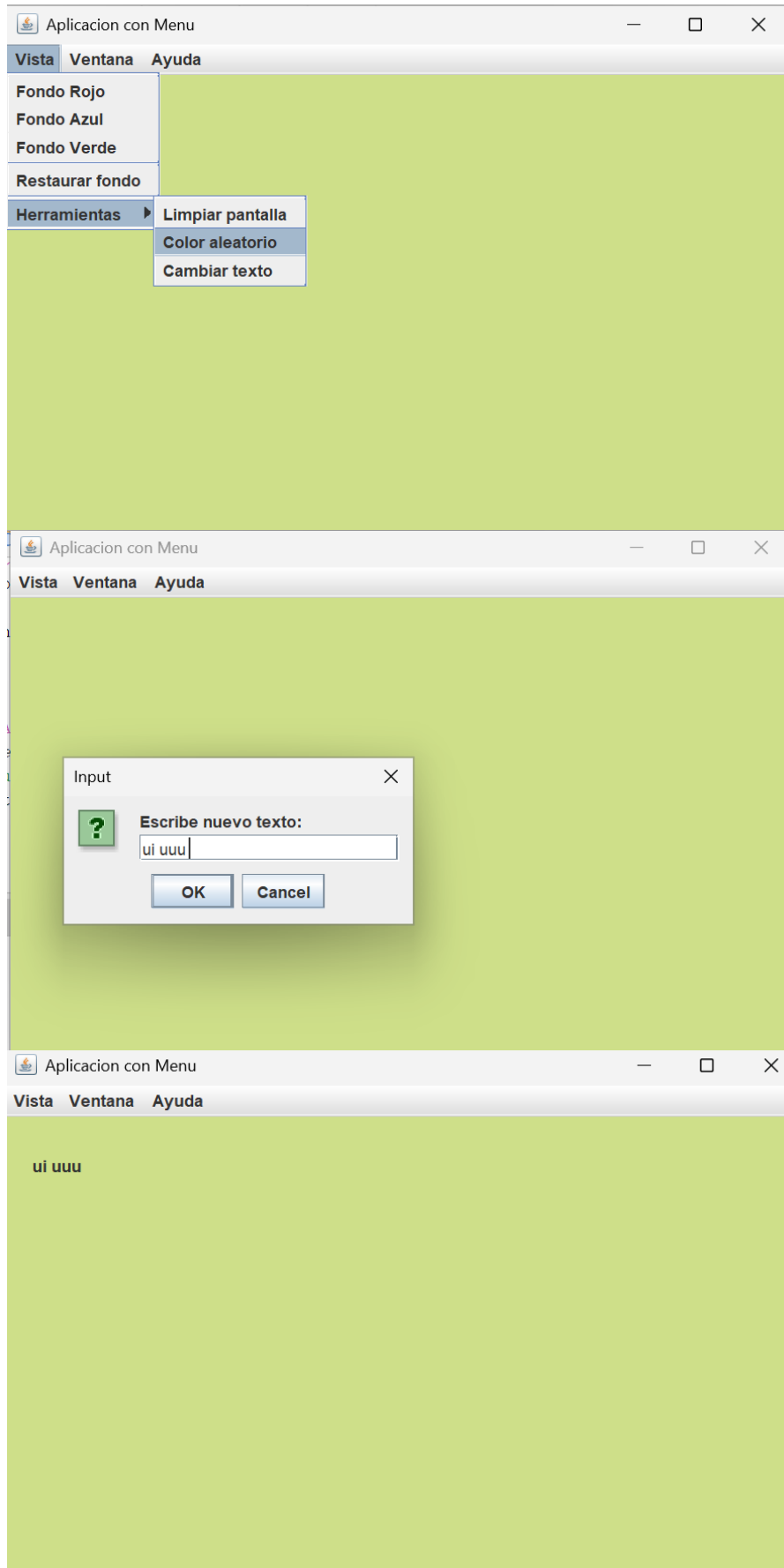
17. Agrega un JMenuItem 'Maximizar' que llame a `setExtendedState(JFrame.MAXIMIZED_BOTH)`.

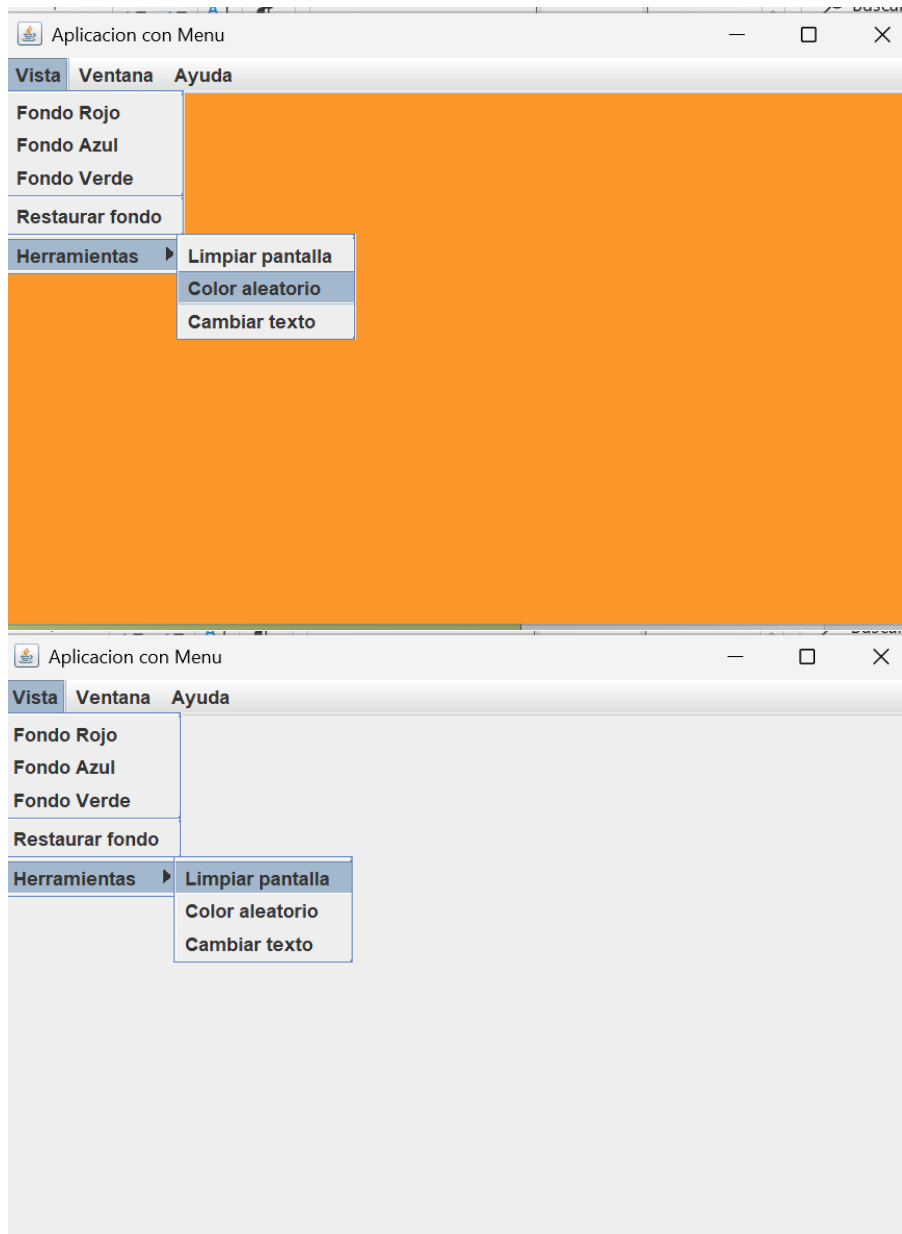


18. Agrega aceleradores de teclado a los items usando `miRojo.setAccelerator(KeyStroke.getKeyStroke("ctrl R"))`.

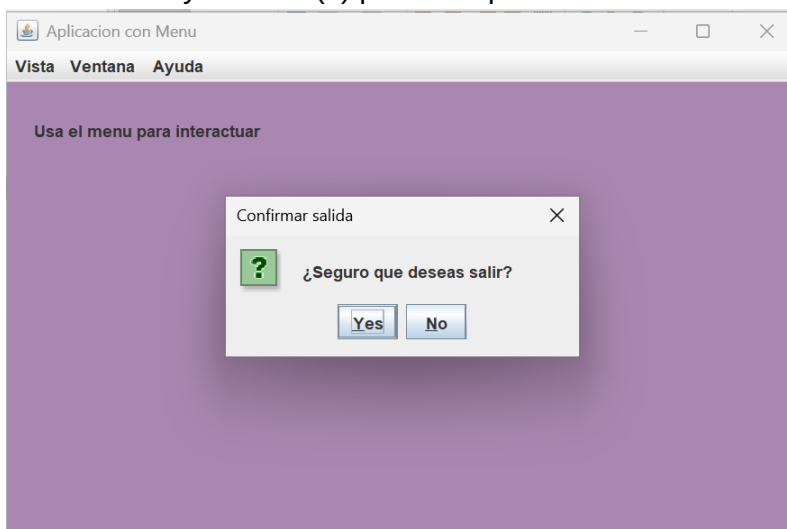


19. Crea un submenu 'Herramientas' dentro de 'Vista' con opciones adicionales.





20. Cambia `System.exit(0)` por un `JOptionPane` de confirmación antes de salir.



7. Practica 6: JCheckBox y JRadioButton

7.1 JCheckBox - Selección Múltiple

JCheckBox es un boton de dos estados (marcado / no marcado) que puede usarse de forma independiente o en conjunto. Permite al usuario activar varias opciones simultaneamente. El metodo isSelected() retorna true si esta marcado.

7.2 JRadioButton - Selección Exclusiva

JRadioButton funciona en grupos mutuamente excluyentes: solo uno puede estar seleccionado a la vez. Para lograr esto se usa ButtonGroup, que coordina la deseleccion automatica.

7.3 Ejercicio Integrador: Cotizador de Servicios

Crea un cotizador donde el usuario seleccione una categoria (JRadioButton) y servicios adicionales (JCheckBox), y al presionar un boton calcule el precio total.

```
import javax.swing.*;
import java.awt.event.*;

public class Cotizador extends JFrame implements ActionListener {

    // Plan base (excluyentes)
    private JLabel lblPlan, lblExtras, lblTotal;
    private JRadioButton rBasico, rProfesional, rEmpresarial;
    private ButtonGroup bgPlanes;

    // Extras (acumulables)
    private JCheckBox chkSoporte, chkBackup, chkSeguridad;
    private JButton btnCotizar;

    public Cotizador() {
        setLayout(null);
        setTitle("Cotizador de Servicios");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        lblPlan = new JLabel("Selecciona tu plan:");
        lblPlan.setBounds(20, 15, 200, 25);
        add(lblPlan);

        bgPlanes = new ButtonGroup();

        rBasico = new JRadioButton("Basico $199/mes");
        rProfesional = new JRadioButton("Profesional $399/mes");
        rEmpresarial = new JRadioButton("Empresarial $799/mes");
        rBasico.setBounds(20, 45, 250, 25);
        rProfesional.setBounds(20, 75, 250, 25);
        rEmpresarial.setBounds(20, 105, 250, 25);
        rBasico.setSelected(true); // Valor por defecto

        bgPlanes.add(rBasico);
```



```
        bgPlanes.add(rProfesional);
        bgPlanes.add(rEmpresarial);
        add(rBasico); add(rProfesional); add(rEmpresarial);

        lblExtras = new JLabel("Servicios adicionales:");
        lblExtras.setBounds(20, 145, 220, 25);
        add(lblExtras);

        chkSoporte = new JCheckBox("Soporte 24/7      +$99");
        chkBackup = new JCheckBox("Backup diario    +$49");
        chkSeguridad = new JCheckBox("Seguridad Plus  +$79");
        chkSoporte.setBounds(20, 175, 250, 25);
        chkBackup.setBounds(20, 205, 250, 25);
        chkSeguridad.setBounds(20, 235, 250, 25);
        add(chkSoporte); add(chkBackup); add(chkSeguridad);

        btnCotizar = new JButton("Ver cotizacion");
        btnCotizar.setBounds(20, 280, 160, 35);
        btnCotizar.addActionListener(this);
        add(btnCotizar);

        lblTotal = new JLabel("Total mensual: $0.00");
        lblTotal.setBounds(20, 330, 300, 30);
        add(lblTotal);
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        int total = 0;

        // Plan base
        if (rBasico.isSelected()) total += 199;
        else if (rProfesional.isSelected()) total += 399;
        else if (rEmpresarial.isSelected()) total += 799;

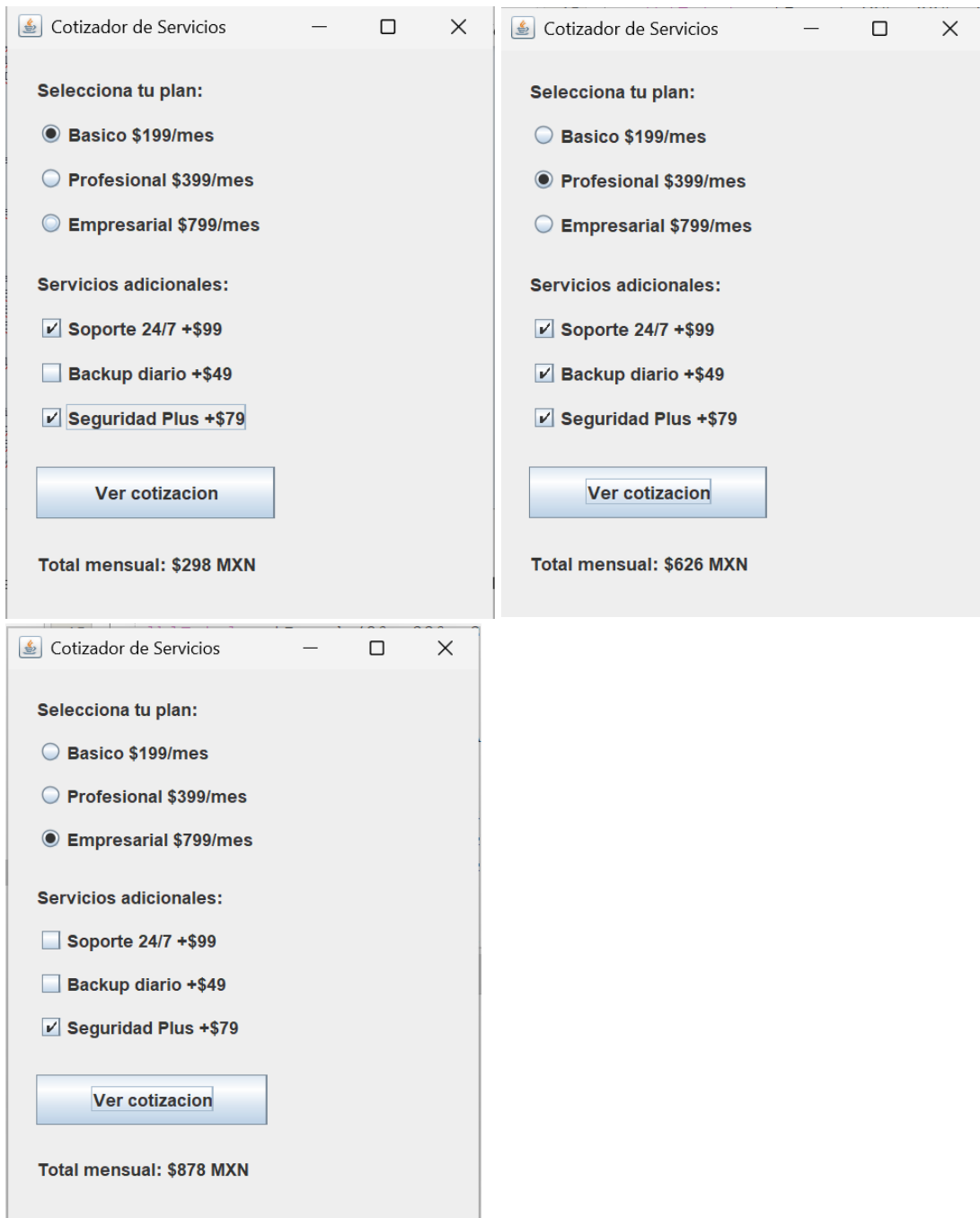
        // Extras seleccionados
        if (chkSoporte.isSelected()) total += 99;
        if (chkBackup.isSelected()) total += 49;
        if (chkSeguridad.isSelected()) total += 79;

        lblTotal.setText(String.format("Total mensual: $%,d MXN", total));
    }

    public static void main(String[] args) {
        Cotizador cot = new Cotizador();
        cot.setBounds(200, 150, 340, 420);
        cot.setVisible(true);
    }
}
```

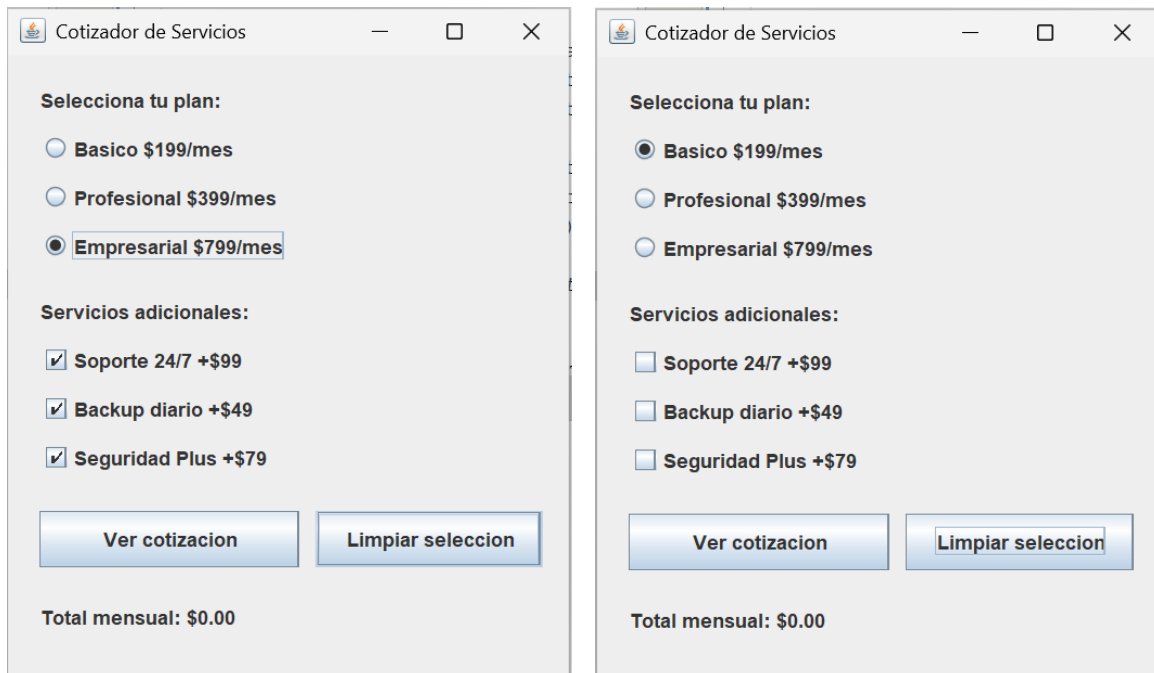
Actividades

21. Ejecuta y selecciona distintas combinaciones de plan y extras. Verifica que el total sea correcto.

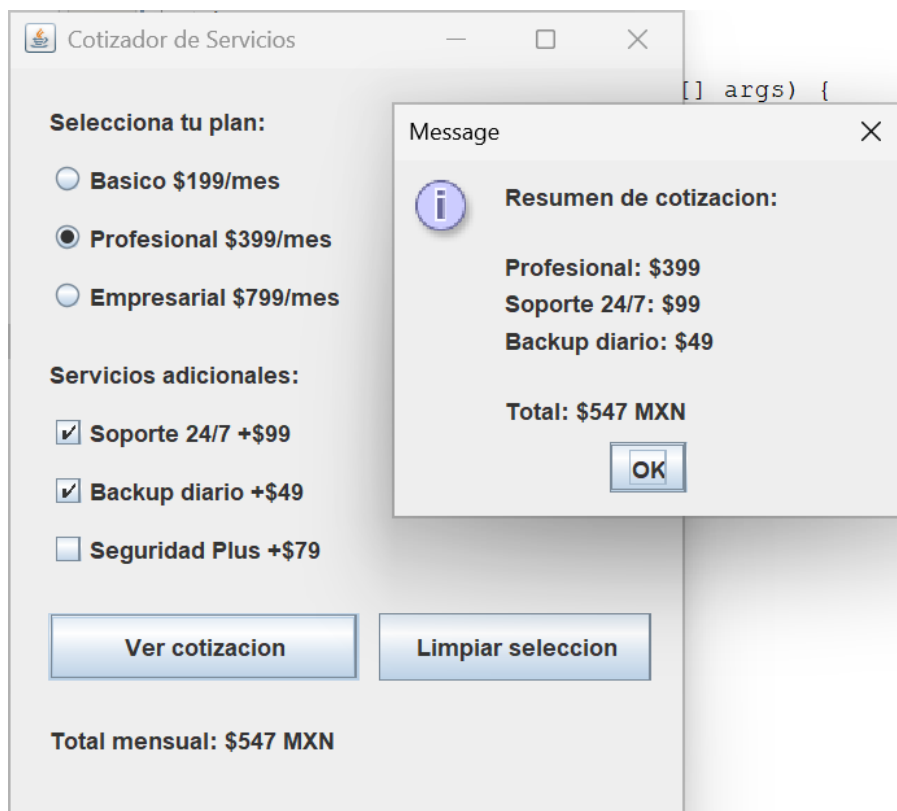


22. Observa que al seleccionar un JRadioButton, el anterior se deselectiona automáticamente.

23. Agrega un boton 'Limpiar seleccion' que restablezca el plan a Basico y desmarque todos los extras.



24. Muestra un resumen detallado con JOptionPane al presionar cotizar, listando cada concepto y su precio.



8. Proyecto Integrador

Sistema de Registro de Estudiantes

Integra todos los componentes aprendidos en esta práctica para construir un sistema de registro de estudiantes con las siguientes características:

Especificaciones del Sistema

- JTextField para capturar: nombre, matricula y promedio.
- JComboBox con las carreras disponibles (al menos 5).
- JRadioButton para seleccionar el turno: matutino, vespertino o nocturno.
- JCheckBox para servicios inscritos: biblioteca, deportes, cafetería.
- JButton 'Registrar' que valide los datos y muestre el registro completo.
- JTextArea donde se acumulen todos los registros realizados en la sesión.
- Barra de menú con opciones: Nuevo registro, Limpiar lista, Exportar (solo mensaje), Salir.

Criterios de Evaluacion

Criterio	Puntos	Obtenido
Interfaz completa con todos los componentes requeridos	20	
Validación de campos vacíos y tipos de dato correctos	20	
Eventos funcionan correctamente en todos los controles	20	
Acumulación de registros en JTextArea	15	
Menú funcional con todas las opciones	15	
Código organizado, comentado y con buenas practicas	10	
TOTAL	100	

Sistema de Registro de Estudiantes

Archivo

Nombre:

Matricula:

Promedio:

Carrera:

Turno: ☐ Matutino ☐ Vespertino ☐ Nocturno

Servicios: ☐ Biblioteca ☐ Deportes ☐ Cafeteria

Sistema de Registro de Estudiantes

Archivo

Nombre:

Matricula:

Promedio:

Carrera:

Turno: ☐ Matutino ☐ Vespertino ☐ Nocturno

Servicios: ☐ Biblioteca ☐ Deportes ☐ Cafeteria

Message

 Completa todos los campos

Sistema de Registro de Estudiantes

Archivo

Nombre:

Matricula:

Promedio:

Carrera:

Turno:

Servicios:

Registrar

Message

Promedio invalido

OK

Sistema de Registro de Estudiantes

Archivo

Nombre:

Matricula:

Promedio:

Carrera:

Turno: ☐ Matutino ☐ Vespertino ☐ Nocturno

Servicios: ☐ Bibliotecario

Registrar

Message

Selecciona un turno

OK

Sistema de Registro de Estudiantes

Archivo

Nombre:

Matricula:

Promedio:

Carrera:

Turno: ☒ Matutino ☐ Vespertino ☐ Nocturno

Servicios: ☒ Biblioteca ☐ Deportes ☒ Cafeteria

Nombre: Andrea | Matricula: 24580011 | Promedio: 9.1 | Carrera: Sistemas | T

Sistema de Registro de Estudiantes

Archivo

Nuevo registro

Limpiar lista

Exportar

Salir

Nombre:

Matricula:

Promedio:

Carrera:

Turno: ☒ Matutino ☐ Vespertino ☐ Nocturno

Servicios: ☒ Biblioteca ☐ Deportes ☒ Cafeteria

Nombre: Andrea | Matricula: 24580011 | Promedio: 9.1 | Carrera: Sistemas | T

Sistema de Registro de Estudiantes

Archivo

Nombre: Lucia

Matricula: 2458008

Promedio: 9.4

Carrera: Sistemas

Turno: Industrial Nocturno

Servicios: Electronica Cafeteria

Administracion

Registrar

dio: 9.1 | Carrera: Sistemas | Turno: Matutino | Servicios: Biblioteca Cafeteria

Sistema de Registro de Estudiantes

Archivo

Nombre: Lucia

Matricula: 2458008

Promedio: 9.4

Carrera: Administracion

Turno: ☐ Matutino ☒ Vespertino ☐ Nocturno

Servicios: ☒ Biblioteca ☒ Deportes ☐ Cafeteria

Registrar

Nombre: Andrea | Matricula: 24580011 | Promedio: 9.1 | Carrera: Sistemas | T
Nombre: Lucia | Matricula: 2458008 | Promedio: 9.4 | Carrera: Administracion |

Sistema de Registro de Estudiantes

Archivo

Nuevo registro

Limpiar lista

Exportar

Salir

Nombre: Lucia

Matricula: 2458008

Promedio: 9.4

Carrera: Administracion

Turno: ☐ Matutino ☒ Vespertino ☐ Nocturno

Servicios: ☒ Biblioteca ☒ Deportes ☐ Cafeteria

Registrar

Nombre: Andrea | Matricula: 24580011 | Promedio: 9.1 | Carrera: Sistemas | T
Nombre: Lucia | Matricula: 2458008 | Promedio: 9.4 | Carrera: Administracion

Sistema de Registro de Estudiantes

Archivo

Nombre: Lucia

Matricula: 2458008

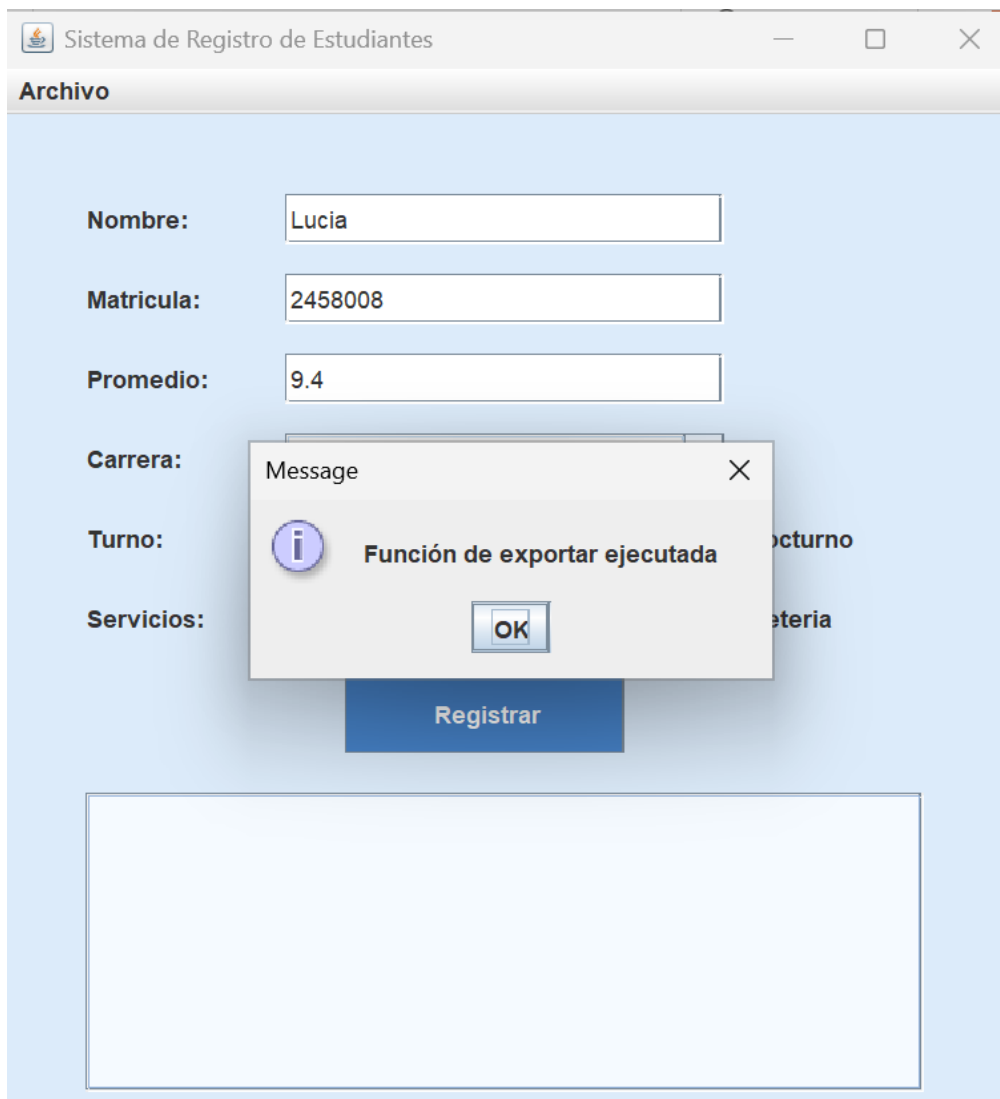
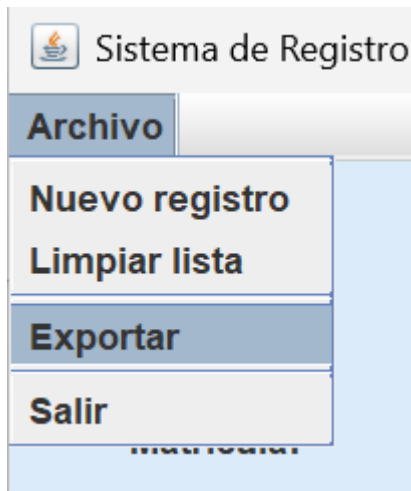
Promedio: 9.4

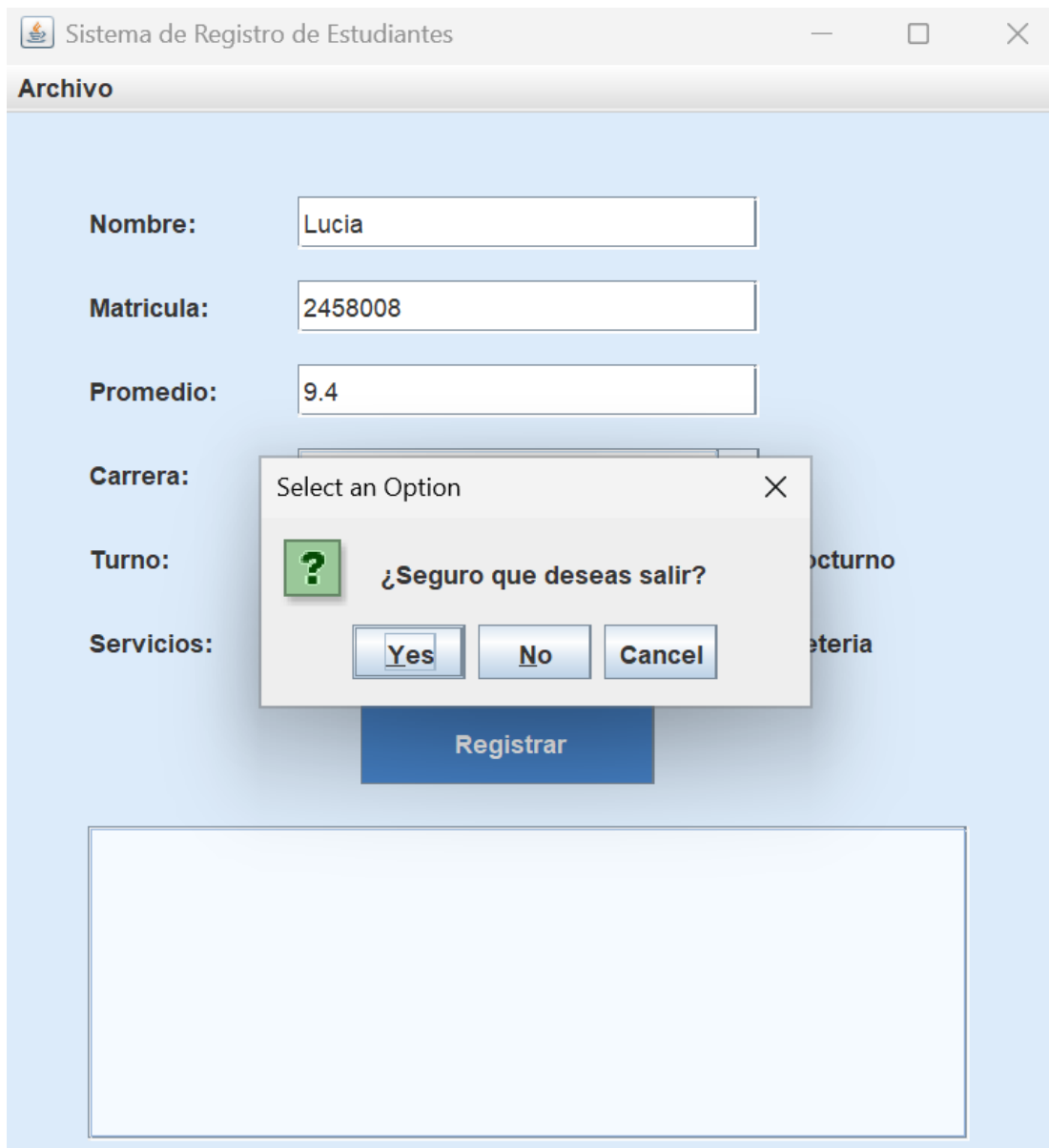
Carrera: Administracion

Turno: ☐ Matutino ☒ Vespertino ☐ Nocturno

Servicios: ☒ Biblioteca ☒ Deportes ☐ Cafeteria

Registrar





9. Apéndice: Referencia Rápida de Componentes

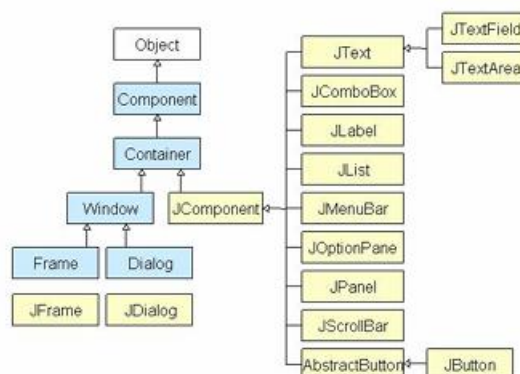
Interface Gráfica de Usuario en Java: Swing Hasta ahora se han resuelto los algoritmos haciendo las salidas a través de una consola en modo texto. La realidad que es muy común la necesidad de hacer la entrada y salida de datos mediante una interfaz más amigable con el usuario. La interfaz de usuario es un aspecto muy importante de cualquier aplicación.

Una aplicación sin un interfaz fácil, impide que los usuarios saquen el máximo rendimiento del programa. Java proporciona los elementos básicos para construir interfaces de usuario a través de AWT y opciones para mejorarlas mediante Swing, que sí permite la creación de interfaces de usuario de gran impacto y sin demasiados quebraderos de cabeza.

Al nivel más bajo, el sistema operativo transmite información desde el ratón y el teclado como dispositivos de entrada. AWT fue diseñado pensando en que el programador no tuviese que preocuparse de detalles como controlar el movimiento del ratón o leer el teclado, ni tampoco atender a detalles como la escritura en pantalla. AWT constituye una librería de clases orientada a objeto para cubrir estos recursos y servicios de bajo nivel.

Cuando se empieza a utilizar Swing, se observa que se ha dado un gran paso adelante respecto a AWT. Ahora los componentes del interfaz gráfico son Beans y utilizan el nuevo modelo de Delegación de Eventos de Java. Swing proporciona un conjunto completo de componentes, todos ellos lightweight, es decir, ya no se usan componentes "peer" dependientes del sistema operativo, y además, Swing está totalmente escrito en Java.

Todo ello redundará en una mayor funcionalidad en manos del programador, y en la posibilidad de mejorar en gran medida la presentación de los interfaces gráficos de usuario. Son muchas las ventajas que ofrece el uso de Swing, por ejemplo, la navegación con el teclado es automática, cualquier aplicación Swing se puede utilizar sin ratón, sin tener que escribir ni una línea de código adicional, las etiquetas de información, o "tool tips", se pueden crear con una sola línea de código. Además, Swing aprovecha la circunstancia de que sus componentes no están renderizados sobre la pantalla por el sistema operativo para soportar lo que llaman "pluggable look and feel", es decir, que la apariencia de la aplicación se adapta dinámicamente al sistema operativo y plataforma en que esté corriendo.



Componente	Metodo principal	Descripcion
JFrame	setBounds / setTitle	Ventana principal de la aplicacion
JLabel	setText / getText	Muestra texto o imagen (solo lectura)
JButton	addActionListener	Boton que dispara eventos de clic
TextField	getText / setText	Campo de texto de una linea editable
TextArea	getText / append	Area de texto multilinea (usar con JScrollPane)
ComboBox	getSelectedItem / addItem	Lista desplegable de seleccion unica
MenuBar	setJMenuBar (mb)	Barra horizontal de menus
Menu	add(JMenuItem)	Menu desplegable con subopciones
MenuItem	addActionListener	Opcion seleccionable dentro de un menu
CheckBox	isSelected	Casilla de seleccion multiple (on/off)
RadioButton	isSelected + ButtonGroup	Opcion exclusiva dentro de un grupo

RECURSOS ADICIONALES: Documentacion oficial Oracle Swing:
<https://docs.oracle.com/javase/tutorial/uiswing/> | IntelliJ IDEA GUI Designer: incluido en la version Community para diseno visual de formularios.